

Zero-Touch Vulnerability Remediation Platform

Enterprise Technical Documentation

AI-Driven Automated Vulnerability Management System

Project Details

Organization	Hewlett Packard Enterprise (HPE)
Project Duration	Feb 2026 – June 2026
Document Type	Technical Documentation

Team Members Sidharth Banthia Abhimanyu Joshi Rahul Raj Prince Jain Vaibhav Samdani	Mentors HPE Mentors Thavamani Raja S. Vivek Chinta College Mentor Dr. Seema Sharma
---	--



JECRCTM
UNIVERSITY
BUILD YOUR WORLD



Hewlett Packard
Enterprise

Table of Contents

Zero-Touch Vulnerability Remediation Platform.....	1
Enterprise Technical Documentation.....	1
AI-Driven Automated Vulnerability Management System.....	1
Table of Contents.....	2
1. Problem Description.....	8
2. Solution Description.....	9
2.1 Workflow Overview.....	9
2.2 What This Replaces.....	9
3. Solution Architecture.....	10
4. Setup Details.....	11
4.1 Amazon S3 Setup.....	11
Step 1: Create S3 Bucket.....	11
Step 2: Configure Bucket Settings.....	11
Purpose in the Project.....	12
4.2 IAM Role Configuration.....	12
Step 1: Create IAM Role.....	12
Step 2: Attach Required Policies.....	12
Step 3: Create Role.....	12
Purpose in the Project.....	13
4.3 EC2 Instance Setup.....	13
Step 1: Launch Instance.....	13
Step 2: Select AMI.....	13
Step 3: Select Instance Type.....	13
Step 4: Create Key Pair.....	13
Step 5: Configure Network Settings.....	13
Step 6: Configure Storage.....	14
Step 7: Attach IAM Role.....	14
Step 8: Launch Instance.....	14
4.4 AWS Systems Manager (SSM) Setup.....	14
Step 1: Verify SSM Agent.....	14
Step 2: Verify Managed Node Registration.....	14
Step 3: Execute Test Command.....	15

4.5 S3 Log Upload Validation.....	15
4.6 IAM User Setup for GitHub Actions.....	15
Step 1: Create User.....	15
Step 2: Attach Policies.....	15
Step 3: Generate Access Keys.....	16
4.7 GitHub Secrets Configuration.....	16
4.8 End-to-End Validation.....	16
4.9 Security Considerations.....	16
4.10 Jira Setup & Integration.....	17
Step 1: Create a Dedicated Jira Project.....	17
Step 2: Configure Workflow States.....	17
Step 3: Generate Jira API Credentials.....	17
Step 4: Configure Environment Variables.....	18
Purpose.....	18
4.11 Vulnerable VM Setup Guide.....	18
Prerequisites.....	18
Trivy Setup for File System Vulnerability Scan on EC2.....	18
Step 1: Connect to the EC2 Instance.....	18
Step 2: Install Trivy.....	18
Step 3: Verify the Installation.....	19
Step 4: Scan the EC2 Filesystem.....	19
Step 5: Verify the Generated Report.....	19
Step 6: Exit the EC2 Instance.....	19
Step 7: Download the Scan Report.....	19
4.12 PostgreSQL Database Setup.....	20
Prerequisites.....	20
Step 1: Create an AWS RDS PostgreSQL Instance.....	20
Step 2: Connect to the RDS Instance.....	21
Step 3: Create the Application Database.....	22
Step 4: Run the Schema.....	22
Step 5: Verify the Setup.....	24
Step 6: Configure the Application Environment.....	25
4.13 Deployment Configuration.....	25
4.13.1 Prerequisites.....	25
4.13.2 Environment Setup.....	25
4.13.3 Repository Setup.....	25
4.13.4 Create and activate a virtual environment.....	25

4.13.5 Python Dependencies.....	26
4.13.6 Node Dependencies.....	26
4.13.7 Database Setup.....	26
4.13.8 ngrok Setup.....	26
4.13.9 Startup Sequence.....	27
4.13.10 Data Migration Routine.....	27
4.13.11 GitHub Webhook Setup for Event-Driven HITL Workflow.....	28
Configure the GitHub Webhook.....	28
4.13 GitHub Personal Access Token Setup (MCP).....	28
Step 1: Open GitHub Token Settings.....	28
Step 2: Configure Token Details.....	28
Step 3: Configure Repository Permissions.....	29
Step 4: Generate Personal Access Token.....	29
Step 5: Environment Configuration.....	29
5. File Structure.....	30
6. Components.....	31
7. Technology Stack.....	32
8. Understanding the Pipeline.....	33
8.1 Parser Agent.....	33
Overview.....	33
Working Principle.....	33
Input.....	34
Output.....	34
Self-Healing Capability.....	34
8.2 Normalization Agent.....	34
Overview.....	34
Working Principle.....	35
Standardized Schema – Core Fields.....	36
Input & Output.....	36
Self-Healing Capability.....	36
8.3 Vulnerability Enrichment Agent – Part 1.....	37
Core Objective.....	37
Execution Workflow (Node-by-Node).....	37
8.4 Vulnerability Enrichment Agent – Part 2.....	38
Core Objective.....	38
Execution Workflow (Node-by-Node).....	39
8.5 Asset Enrichment Agent – Part 1.....	41

Overview: Why Two Separate Agents.....	41
Input & Output.....	41
Pipeline Structure.....	41
Node 1 — Load & Validate CSV.....	41
DB Lookup — Priority-Ordered Matching.....	42
Enriching Existing Assets.....	42
New Assets — UUID Generation & Skeleton Insert.....	42
Batch Operations Throughout.....	42
Retry Logic & Transient Error Handling.....	43
Why Everything Is Async.....	43
Control Transfer to Agent 2.....	43
8.6 Asset Enrichment Agent – Part 2.....	43
Input & Output.....	43
What Gets Skipped & Why.....	43
Inference Pipeline — Deterministic First.....	44
LLM Fallback — Only When Deterministic Inference Hits a Wall.....	44
Concurrency Within the Node.....	45
DB Upsert — One Round Trip for All Assets.....	45
Writing Back to CSV.....	45
8.7 Asset Criticality Agent.....	46
8.8 Prioritization Agent.....	46
Core Objective.....	46
Execution Workflow (Node-by-Node).....	46
8.9 Remediation Agent.....	48
1. Workflow Initialization.....	48
2. Vulnerability Research.....	49
3. Script Generation.....	49
4. GitHub Automation.....	49
5. Validation in a Sandboxed Environment.....	49
6. Human-in-the-Loop (HITL) Review.....	50
7. Workflow Completion.....	50
9. Database Schema.....	51
9.1 vulnerabilities.....	51
9.2 vulnerability_intel.....	51
9.3 assets.....	52
9.4 asset_vulnerabilities.....	52

9.5 vulnerabilities_history.....	53
9.6 Dataset Description.....	54
10. Cron Jobs.....	55
10.1 NVD Sync — Cron Job.....	55
Why This Exists.....	55
What It Does.....	55
Why Static Data Stays Local.....	55
What Lives Here vs. vulnerability_intel.....	55
10.2 Vulnerability Intelligence Cron Job.....	56
Why It Exists.....	56
What It Does.....	56
Why Intel Data Stays Local.....	56
What Lives Here vs. The Vulnerability Agent Cron Job.....	57
11. Dashboard.....	58
11.1 Dashboard Metrics.....	58
Average Cost per Vulnerability.....	58
Average MTTR (Mean Time To Remediate).....	58
Total Vulnerabilities.....	58
Total Solved Vulnerabilities.....	58
Tokens Used & Historical Token Consumption.....	58
Success Rate.....	58
Vulnerabilities by Severity.....	58
Fleet Distribution / Recent Activity.....	58
12. Agent Performance Evaluation.....	60
12.1 Evaluation Objectives.....	60
12.2 Categorized Key Performance Indicators (KPIs).....	61
Core System Metrics.....	61
12.3 Measurement Methodology & Mathematical Formulas.....	62
Resolution & Trust Formulations.....	62
Efficiency & Latency Formulations.....	62
Token and Cost Consumption.....	62
13. Full Integration Configuration.....	63
13.1 System Architecture.....	63
13.2 Presentation Tier (Frontend).....	63
13.3 Orchestration Tier (Backend).....	63
13.4 Persistence Tier (Databases).....	63

13.5 Multi-Agent Orchestration.....	64
13.5.1 Agent Topology.....	64
13.5.2 State Management & Human-in-the-Loop.....	64
14. Errors / Challenges Faced.....	65
15. Conclusion.....	66
Platform Transformation.....	66
Scalability.....	66
Reduced Mean-Time-To-Respond (MTTR).....	66
Local Threat Intelligence.....	66
Traceability.....	66
16. Future Scope.....	67
17. Member Contributions.....	68
18. References & Resources.....	69

1. Problem Description

Modern infrastructure teams manage hundreds to thousands of virtual machines, each continuously exposed to newly disclosed security vulnerabilities. Despite having scanning tools in place, the end-to-end process of going from a discovered vulnerability to a verified fix remains entirely manual.

The current model operates in four sequential human-driven stages:

- **Manual Triage:** Analysts sift through raw scanner output across multiple tools, each with a different format and schema. There is no systematic enrichment, no asset's business context (criticality) weighting, and no exploit-likelihood scoring. Priority decisions lead to inconsistency and missed high-risk findings.
- **Manual Ticket Creation:** Once triaged, a Jira ticket is hand-written for each finding. This is repetitive, slow, and introduces variability in ticket quality and structure across analysts.
- **Manual Fix Generation and Updation:** Remediation engineers research each vulnerability independently, write scripts without a standardized testing environment, apply fixes directly to production, and manually update ticket status throughout. There is no feedback mechanism when a fix fails.
- **Manual Validation:** Post-fix verification is either skipped or performed informally. No before-and-after evidence is captured, leaving no audit trail and no reliable confirmation that the vulnerability was actually resolved.

The result is a slow, inconsistent, and unauditable process. Mean time to remediate stretches into weeks. Backlogs accumulate. Critical assets are not differentiated from low-value ones. Engineers spend the majority of their time on work that should not require human judgment.

2. Solution Description

The Automated Vulnerability Management System replaces the manual four-stage model with an end-to-end automated pipeline, retaining a single human approval gate before any change reaches production.

2.1 Workflow Overview

The system executes the following sequence automatically:

- Multiple security scanners are given into a system where they are parsed and normalized into a unified schema, with deduplication applied across scanner outputs.
- Each finding is enriched using local replicas of NVD, EPSS, and CISA KEV, and scored by a prioritization engine that factors in asset criticality alongside exploit likelihood. High-Low-Priority-wise findings enter the remediation queue immediately.
- For each queued vulnerability, the system clones the target VM, creates a Jira ticket and a linked GitHub branch, and generates a tailored remediation script (`remediate.sh`).
- The script is pushed to GitHub, triggering a CI/CD pipeline that executes it on the cloned VM. If execution fails or the vulnerability persists, a self-correction agent analyzes the error logs, revises the script, and retries automatically.
- On successful remediation, before-and-after evidence is captured and a Pull Request is raised. A human reviewer approves or rejects the PR. On approval, the fix is applied to the production VM, the Jira ticket is resolved, and the record is closed. On rejection, the agent, with human feedback, regenerates the remediation script and the remediation pipeline reruns.

2.2 What This Replaces

- Manual triage is replaced by automated normalization and industry standard formulae-based prioritization.
- Manual ticket creation is replaced by structured, auto-generated Jira tickets with full enrichment context.
- Manual fix generation is replaced by an agent that produces and iteratively corrects remediation scripts.
- Manual validation is replaced by automated post-execution verification with evidence capture. The only human step retained is the final PR approval before production deployment. Every other stage runs without manual intervention.

3. Solution Architecture

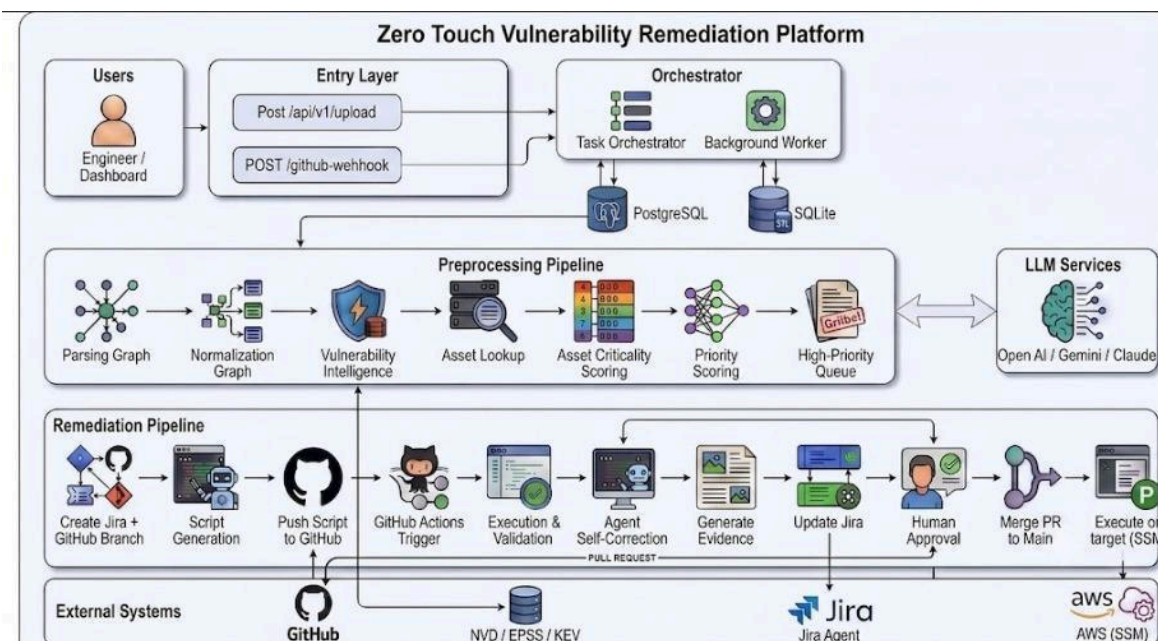


Figure 1.1: Solution Workflow – End-to-End Automated Vulnerability Remediation Pipeline

4. Setup Details

To enable automated vulnerability remediation in a secure and scalable manner, an AWS-based execution environment was established. The infrastructure was designed to support remote command execution, centralized log storage, secure service-to-service communication, and GitHub-triggered automation workflows.

The overall execution flow is illustrated below:

```
GitHub Actions → AWS Systems Manager (SSM) → EC2 Instance → Execute Remediation Script → Upload Logs to S3 → AI Agent Analysis
```

The setup process consisted of four major components: Amazon S3, IAM Roles and Policies, Amazon EC2, and AWS Systems Manager (SSM).

4.1 Amazon S3 Setup

Amazon S3 was configured as the centralized storage layer for remediation execution logs, validation reports, and workflow artifacts.

Step 1: Create S3 Bucket

Navigate to:

AWS Console → Amazon S3 → Create Bucket

Configure:

Bucket Name: remediation-logs-bucket

Region: Same region as EC2 instance (e.g., ap-south-1)

Step 2: Configure Bucket Settings

The following settings were applied:

- Block Public Access: Enabled
- Versioning: Optional
- Encryption: Enabled

Purpose in the Project

The S3 bucket stores:

- Remediation execution logs
- Validation results
- Workflow outputs
- Evidence generated after remediation

This enables centralized storage and future analysis by AI agents.

4.2 IAM Role Configuration

An IAM Role was created to provide the EC2 instance with the permissions required to communicate with AWS Systems Manager and Amazon S3.

Step 1: Create IAM Role

Navigate to:

IAM → Roles → Create Role

Trusted Entity: AWS Service

Use Case: EC2

Step 2: Attach Required Policies

The following policies were attached:

AmazonSSMManagedInstanceCore

Provides: Systems Manager connectivity, Remote command execution, Managed node registration

AmazonS3FullAccess

Provides: Upload logs to S3, Retrieve remediation artifacts, Store validation reports

Step 3: Create Role

Example role name:

EC2-Remediation-SSM-Role

Purpose in the Project

This role allows the EC2 instance to:

- Receive commands through SSM
- Upload execution logs to S3
- Operate without storing AWS credentials locally

4.3 EC2 Instance Setup

A dedicated EC2 instance was provisioned to execute remediation scripts and validation tasks.

Step 1: Launch Instance

AWS Console → EC2 → Launch Instance

Instance Name: remediation-runner

Step 2: Select AMI

Recommended image:

Ubuntu Server 24.04 LTS

Step 3: Select Instance Type

For development and testing:

t2.micro

Step 4: Create Key Pair

remediation-key.pem

The key pair is used for optional SSH access during development.

Step 5: Configure Network Settings

Create a security group.

Inbound Rules

SSH Port: 22 Source: 0.0.0.0/0

Outbound Rules

Allow All Traffic

Required for: Systems Manager communication, S3 access, Package installations, External downloads

Step 6: Configure Storage

Recommended: 20 GB

Additional storage ensures sufficient space for scanner outputs, temporary remediation files, and validation artifacts.

Step 7: Attach IAM Role

Under Advanced Details → IAM Instance Profile, select:

```
EC2-Remediation-SSM-Role
```

This step is mandatory because SSM communication depends on the attached role.

Step 8: Launch Instance

Launch the EC2 instance and verify that it is running successfully.

4.4 AWS Systems Manager (SSM) Setup

AWS Systems Manager was used to enable secure remote execution without requiring direct SSH access.

Step 1: Verify SSM Agent

Most Ubuntu AMIs include the SSM Agent by default.

Verify installation:

```
snap services amazon-ssm-agent
```

If the agent is not installed:

```
sudo snap install amazon-ssm-agent --classic
```

Enable and start the service:

```
sudo systemctl enable snap.amazon-ssm-agent.amazon-ssm-agent.service
sudo systemctl start snap.amazon-ssm-agent.amazon-ssm-agent.service
```

Step 2: Verify Managed Node Registration

Navigate to:

```
AWS Systems Manager → Fleet Manager
```

or

```
AWS Systems Manager → Managed Nodes
```

Verify that the EC2 instance appears in the managed node list.

Step 3: Execute Test Command

Navigate to:

Systems Manager → Run Command

Select: AWS-RunShellScript

Test commands:

```
echo "Hello from SSM"
hostname
whoami
```

Execute the command and verify successful completion.

4.5 S3 Log Upload Validation

To verify integration between SSM and S3:

- Open Run Command.
- Enable S3 Output.
- Select the remediation logs bucket.
- Execute a test command.

Expected Result:

```
remediation-runs/
```

folders should be created automatically inside the bucket.

This confirms successful log storage and artifact collection.

4.6 IAM User Setup for GitHub Actions

To allow GitHub Actions to trigger AWS services securely, a dedicated IAM User was created.

Step 1: Create User

IAM → Users → Create User

github-actions-remediation

Step 2: Attach Policies

The following permissions were assigned:

```
AmazonSSMFullAccess AmazonS3FullAccess AmazonEC2ReadOnlyAccess
```

Step 3: Generate Access Keys

Generate: Access Key ID and Secret Access Key

These credentials are later stored as GitHub Secrets.

4.7 GitHub Secrets Configuration

Navigate to:

GitHub Repository → Settings → Secrets and Variables → Actions

Configure the following secrets:

```
AWS_ACCESS_KEY_ID
AWS_SECRET_ACCESS_KEY
AWS_REGION
EC2_INSTANCE_ID
REMEDIATION_LOGS_S3_BUCKET
```

These secrets allow GitHub Actions to authenticate with AWS and trigger remediation workflows.

4.8 End-to-End Validation

After infrastructure setup, a complete validation test was performed.

The workflow executed as follows:

- GitHub Actions triggered.
- AWS authentication succeeded.
- SSM command was sent to EC2.
- Remediation script executed successfully.
- Execution logs were uploaded to S3.
- Workflow status was returned.
- AI agents were able to access generated logs for analysis.

This confirmed successful integration between GitHub Actions, IAM, SSM, EC2, and Amazon S3.

4.9 Security Considerations

Once Systems Manager functionality was verified, SSH access was no longer required for day-to-day operations.

As a security best practice:

- SSH access can be disabled.

- All remote execution should be performed through AWS Systems Manager.
- IAM permissions should follow the principle of least privilege.
- S3 bucket access should be restricted to authorized roles only.

This approach improves security while maintaining complete automation capabilities.

4.10 Jira Setup & Integration

Jira was integrated into the vulnerability remediation platform to automate issue tracking, approval workflows, and remediation status management.

Step 1: Create a Dedicated Jira Project

Create a dedicated Jira project for vulnerability management. This project serves as the central repository for all vulnerability tickets generated by the remediation platform.

Step 2: Configure Workflow States

Configure a custom workflow to represent the complete vulnerability remediation lifecycle with the following states:

- **Open** – A new vulnerability ticket is created.
- **Human-in-the-Loop (HITL)** – The ticket waits for manual validation or approval by a security analyst.
- **Resolved** – The vulnerability has been successfully remediated and verified.
- **Unresolved** – The vulnerability cannot be remediated or requires further investigation.

Step 3: Generate Jira API Credentials

Generate the required API credentials to enable secure communication between the remediation platform and Jira.

Required configuration:

- Jira Base URL
- Jira Email ID
- Jira API Token
- Jira Project Key

Step 4: Configure Environment Variables

Store all Jira credentials securely as environment variables instead of hardcoding them in the application.

```
JIRA_BASE_URL=https://your-company.atlassian.net
JIRA_EMAIL=your-email@example.com
JIRA_API_TOKEN=xxxxxxxxxxxxxxxxxxxx
JIRA_PROJECT_KEY=KAN
```

Purpose

Jira serves as the centralized tracking and approval system for the Zero-Touch Vulnerability Remediation Platform. It provides end-to-end visibility into vulnerability status, supports Human-in-the-Loop (HITL) approvals, and maintains a complete audit trail of remediation activities.

4.11 Vulnerable VM Setup Guide

Automated Vulnerability Management System | Version 1.0

Prerequisites

- An active AWS account with EC2 permissions.
- An existing Key Pair in AWS (or create one during launch). Keep the .pem file accessible.
- Your local machine's public IP for SSH access.

Trivy Setup for File System Vulnerability Scan on EC2

To perform vulnerability assessment on the EC2 instance, the Trivy filesystem scanner was installed directly on the server. The following steps were used to install Trivy, generate a vulnerability report in JSON format, and download the report for further processing by the agentic workflow.

Step 1: Connect to the EC2 Instance

SSH into the EC2 instance from the local machine using the private key:

```
ssh -i <your-key.pem> ubuntu@<EC2_PUBLIC_IP>
```

Example:

```
ssh -i mykey.pem ubuntu@13.233.xxx.xxx
```

Step 2: Install Trivy

Update the package list and install the required dependencies:

```
sudo apt update
sudo apt install -y wget gnupg lsb-release
```

Add the official Trivy repository and install Trivy:

```
wget -qO - https://aquasecurity.github.io/trivy-repo/deb/public.key | \
  gpg --dearmor | \
  sudo tee /usr/share/keyrings/trivy.gpg > /dev/null

echo "deb [signed-by=/usr/share/keyrings/trivy.gpg]
https://aquasecurity.github.io/trivy-repo/deb $(lsb_release -sc) main" | \
sudo tee /etc/apt/sources.list.d/trivy.list

sudo apt update
sudo apt install -y trivy
```

Step 3: Verify the Installation

Confirm that Trivy has been installed successfully:

```
trivy --version
```

Step 4: Scan the EC2 Filesystem

Run a filesystem scan and save the results in JSON format:

```
trivy fs / -f json -o new_trivy_scan.json
```

This command scans the entire filesystem of the EC2 instance and generates a structured JSON report named `new_trivy_scan.json` in the current working directory (typically `/home/ubuntu`). Depending on the size of the filesystem, the scan may take several minutes.

Step 5: Verify the Generated Report

Check that the scan report has been created successfully:

```
ls
```

The output should include:

```
new_trivy_scan.json
```

Step 6: Exit the EC2 Instance

```
exit
```

Step 7: Download the Scan Report

From the local machine, copy the generated JSON report using SCP:

```
scp -i hpe-project-key.pem ubuntu@32.197.35.36:/home/ubuntu/new_trivy_scan.json
.
```

This downloads the vulnerability report to the current local directory, where it can be consumed by the vulnerability discovery and remediation workflow.

4.12 PostgreSQL Database Setup

Automated Vulnerability Management System | Version 1.0

Prerequisites

Before starting, ensure you have the following:

- An active AWS account with permissions to create RDS instances.
- Access to a SQL client — pgAdmin, DBeaver, or psql (any one is sufficient).
- Your local machine's public IP address (required for security group configuration).

Note: This guide uses PostgreSQL on AWS RDS. If you prefer a local PostgreSQL installation, skip to Step 2.

Step 1: Create an AWS RDS PostgreSQL Instance

1.1 Navigate to RDS

AWS Console → Services → RDS → Databases → Create Database

1.2 Configure the Database Engine

- Creation method: Standard Create
- Engine: PostgreSQL
- Version: Latest Stable (e.g., PostgreSQL 16.x)

1.3 Database Settings

```
DB Instance Identifier: zero-touch-db
Master Username       : postgres
Master Password       : <your-secure-password>
```

1.4 Instance Size and Storage

```
Instance Class: db.t3.micro
Storage Type   : General Purpose SSD (gp2)
Allocated      : 20 GB
Auto Scaling   : Enabled (optional)
```

Note: For production use, upgrade to db.t3.medium or higher based on expected load.

1.5 Connectivity and Security Group

- Set Public Access to Yes.
- Under VPC Security Groups, create a new security group or select an existing one.
- Add the following inbound rule to the security group:

```
Type      : PostgreSQL
Protocol: TCP
Port      : 5432
Source    : <Your IP Address>/32
```

Note: Setting Source to 0.0.0.0/0 opens the port to all IPs. Only do this for temporary testing. Always restrict to specific IPs.

1.6 Create the Instance

Click Create Database. Provisioning typically takes 5 to 10 minutes. Once the status shows Available, open the instance details and note the following values — you will need them in the next steps:

```
Endpoint  : zero-touch-db.xxxxxxx.ap-south-1.rds.amazonaws.com
Port      : 5432
Username  : postgres
Password  : <your-secure-password>
```

Step 2: Connect to the RDS Instance

Use any of the three options below to connect to your RDS instance.

Option A — pgAdmin

- Open pgAdmin and right-click Servers → Register → Server.
- Under the General tab, set Name to zero-touch-db.
- Under the Connection tab, fill in: Host: <your-rds-endpoint>, Port: 5432, Username: postgres, Password: <your-password>.
- Click Save. The server should appear in the left panel.

Option B — DBeaver

- Click New Database Connection → select PostgreSQL.
- Fill in the same Host, Port, Username, and Password values as above.
- Click Test Connection to verify, then click Finish.

Option C — psql (Terminal)

Run the following command in your terminal:

```
psql -h <your-rds-endpoint> -U postgres -p 5432
```

Enter your password when prompted.

Step 3: Create the Application Database

Once connected, run the following command to create the application database:

```
CREATE DATABASE zero_touch;
```

After creation, switch the active connection to the new database:

```
-- In psql: \c zero_touch
```

```
-- In pgAdmin / DBeaver: Disconnect and reconnect with 'zero_touch' as the database name.
```

Step 4: Run the Schema

With the zero_touch database selected, execute the full schema below. This creates all required tables, indexes, and extensions in the correct order.

Note: Copy the entire block below and run it as a single execution. Do not split it across separate runs, as some tables have foreign key dependencies on others.

```
CREATE EXTENSION IF NOT EXISTS pgcrypto;
```

-- ASSETS

```
CREATE TABLE assets (  
    asset_id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
    ip_address        INET,  
    hostname          VARCHAR(255),  
    instance_id       VARCHAR(255),  
    inferred_role     VARCHAR(100),  
    exposure_score    NUMERIC(5,2) DEFAULT 0.00,  
    environment       VARCHAR(20)  DEFAULT 'unknown',  
    cloud_type        VARCHAR(50),  
    dependency_level  VARCHAR(20)  DEFAULT 'medium',  
    dependency_score   NUMERIC(4,2),  
    asset_criticality_score NUMERIC(5,2) DEFAULT 0.00,  
    first_seen        TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,  
    last_seen         TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP  
);
```

```
CREATE UNIQUE INDEX assets_uniq_hostname      ON assets(hostname)      WHERE
hostname      IS NOT NULL;
CREATE UNIQUE INDEX assets_uniq_instance_id ON assets(instance_id) WHERE
instance_id IS NOT NULL;
CREATE UNIQUE INDEX assets_uniq_ip           ON assets(ip_address)  WHERE
ip_address    IS NOT NULL;
```

-- VULNERABILITIES

```
CREATE TABLE vulnerabilities (
    vuln_id      VARCHAR(100) PRIMARY KEY,
    title        VARCHAR(500) NOT NULL,
    description   TEXT,
    cvss_score    NUMERIC(4,2),
    cvss_vector   VARCHAR(255),
    fix_version   VARCHAR(100),
    published_date DATE
);
```

-- VULNERABILITY INTEL

```
CREATE TABLE vulnerability_intel (
    vuln_id      VARCHAR(100) PRIMARY KEY
                        REFERENCES vulnerabilities(vuln_id) ON DELETE CASCADE,
    epss_score    NUMERIC(5,4),
    kev_flag      INTEGER DEFAULT 0,
    exploit_exists INTEGER DEFAULT 0,
    exploit_count INTEGER DEFAULT 0,
    last_updated  TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP
);
```

-- ASSET VULNERABILITIES

```
CREATE TABLE asset_vulnerabilities (
    asset_id      UUID          NOT NULL,
    vuln_id      VARCHAR(100) NOT NULL,
    severity_raw  VARCHAR(50),
    severity_score NUMERIC(4,2),
    component     VARCHAR(255),
    affected_version VARCHAR(100),
    port          INTEGER,
    fix_available  INTEGER  DEFAULT 0,
    first_seen    TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    last_seen     TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    priority_score NUMERIC(5,2),
    priority_level VARCHAR(20),
```

```
PRIMARY KEY (asset_id, vuln_id),
CONSTRAINT fk_asset FOREIGN KEY (asset_id) REFERENCES assets(asset_id)
ON DELETE CASCADE,
CONSTRAINT fk_vuln FOREIGN KEY (vuln_id) REFERENCES
vulnerabilities(vuln_id) ON DELETE CASCADE
);
```

```
CREATE INDEX IF NOT EXISTS idx_asset_vuln_asset_id ON
asset_vulnerabilities(asset_id);
CREATE INDEX IF NOT EXISTS idx_asset_vuln_vuln_id ON
asset_vulnerabilities(vuln_id);
```

-- VULNERABILITIES HISTORY

```
CREATE TABLE vulnerabilities_history (
    thread_id VARCHAR(50) NOT NULL PRIMARY KEY,
    asset_id VARCHAR(50) NOT NULL,
    cve_id VARCHAR(20),
    score NUMERIC(4,2),
    resolved BOOLEAN DEFAULT FALSE,
    cost NUMERIC(10,2) DEFAULT 0.00,
    token INT DEFAULT 0,
    start_time TIMESTAMPTZ,
    end_time TIMESTAMPTZ,
    time_taken INTERVAL
);
```

Step 5: Verify the Setup

After running the schema, verify that all four tables were created successfully by running:

```
-- psql \dt
```

You should see the following five tables listed:

- assets
- vulnerabilities
- vulnerability_intel
- asset_vulnerabilities
- vulnerabilities_history

If all five are present, the database setup is complete.

Step 6: Configure the Application Environment

Add the following environment variables to your project's .env file so the application can connect to the database:

```
DB_HOST      = <your-rds-endpoint>
DB_PORT      = 5432
DB_NAME      = zero_touch
DB_USER      = postgres
DB_PASSWORD  = <your-secure-password>
```

Note: Never commit your .env file to version control. Add it to .gitignore before your first commit.

4.13 Deployment Configuration

4.13.1 Prerequisites

- Node.js: v22.13.1+ (with npm 11.7.0+)
- Python: v3.13+
- PostgreSQL: v14+ (Local or Cloud)
- ngrok

4.13.2 Environment Setup

Create a .env file in the root directory by duplicating .env.example. Provide the secure parameters.

4.13.3 Repository Setup

Clone the repository and set up a Python virtual environment. Virtual environments ensure that project dependencies are isolated and do not conflict with system-wide Python packages.

Clone the repository:

```
git clone
https://github.com/Rahul-Data-Scientist/Zero-Touch-Vulnerability-Remediation
cd Zero-Touch-Vulnerability-Remediation
```

4.13.4 Create and activate a virtual environment

Windows:

```
python -m venv venv
./venv\Scripts\activate
```

macOS/Linux:

```
python3 -m venv venv
source venv/bin/activate
```

4.13.5 Python Dependencies

The backend relies on several asynchronous and AI-focused libraries.

Install the backend dependencies:

```
cd backend
pip install -r requirements.txt
```

4.13.6 Node Dependencies

The frontend uses Next.js and TailwindCSS.

Install the frontend dependencies:

```
cd ../frontend
npm install
```

Commands:

- Development: `npm run dev`
- Production Build: `npm run build` followed by `npm run start`

4.13.7 Database Setup

The system implements a dual-database architecture:

- SQLite (`state_db.sqlite`): Used as a local, temporary queue and LangGraph checkpoint. It handles active workflows.
- PostgreSQL: Used for long-term storage, dashboard analytics, and historical metrics.

Setup Steps:

- Ensure PostgreSQL is installed and running locally.
- Create a database matching your `DB_NAME` environment variable.
- The backend handles initialization automatically on startup via `migrate_sqlite_to_psql` and internal table creation routines.

4.13.8 ngrok Setup

Because the system interacts with GitHub Actions and Pull Requests, GitHub must be able to send webhook events back to your local environment.

1. Install & Authenticate ngrok:

Download ngrok, install it, and add your auth token:

```
ngrok config add-authtoken <your_token>
```

2. Start the tunnel:

```
ngrok http 8000
```

Note the forwarding URL (e.g., <https://abcdef.ngrok-free.app>) and configure it in your GitHub repository's Webhook settings targeting the `/github-webhook` route.

4.13.9 Startup Sequence

To prevent connection refusals and ensure smooth initialization, start the services in this exact order:

- Start ngrok: `ngrok http 8000` (Needed before backend so you know your webhook URL).

Backend Initialization:

```
cd backend
uvicorn main:app --reload --port 8000 \
  --reload-exclude "normalized_output/*" \
  --reload-exclude "parsed_csvs/*" \
  --reload-exclude "raw_scanner_outputs/*" \
  --reload-exclude "state_db.sqlite"
```

Or:

```
cd backend
uvicorn main:app --port 8000
```

The backend will automatically initialize SQLite, connect to Postgres, and spawn background tasks.

Frontend Initialization:

```
cd frontend
npm run dev
```

Access: Open your browser to <http://localhost:3000>

4.13.10 Data Migration Routine

To manually migrate resolved tasks from the volatile SQLite queue to the persistent PostgreSQL warehouse, execute the migration utility:

```
cd backend
python utils/migrate.py
```

4.13.11 GitHub Webhook Setup for Event-Driven HITL Workflow

To avoid keeping the agent running while waiting for human approval, an **event-driven HITL workflow** was implemented using GitHub Webhooks. Instead of continuously checking for feedback, the agent resumes only when GitHub sends a webhook event.

Configure the GitHub Webhook

Go to **Repository** → **Settings** → **Webhooks** → **Add webhook** and configure:

- **Payload URL:** `https://<your-free-domain>/github-webhook`
- **Content Type:** `application/json`
- **Events:**
 - Issue comments
 - Pull requests
 - Pull request reviews

After saving the webhook, GitHub automatically sends these events to the FastAPI endpoint, allowing the agent to continue its workflow only when human feedback is received.

4.13 GitHub Personal Access Token Setup (MCP)

To enable the MCP GitHub Server to securely access and manage GitHub repositories, a Fine-Grained Personal Access Token (PAT) was created with the required repository permissions.

Step 1: Open GitHub Token Settings

Navigate to:

GitHub → **Settings** → **Developer Settings** → **Personal Access Tokens** → **Fine-grained Tokens**

Click **Generate New Token**.

Step 2: Configure Token Details

Provide an appropriate token name.

Under **Repository Access**, select:

- **Only Select Repositories**
- Choose the repository on which the MCP agent will perform operations.

Step 3: Configure Repository Permissions

Click **Add Permissions** and grant the following permissions:

- Actions
- Commit Statuses
- Contents
- Pull Requests
- Workflows

Keep **Metadata** permission enabled (default).

For all selected permissions (except Metadata), change the access level to:

Read and Write

Step 4: Generate Personal Access Token

Review the configuration and click **Generate Token**.

Copy and securely store the generated token, as it will only be displayed once.

Step 5: Environment Configuration

Store the generated Personal Access Token in the project's **.env** file using the following environment variable:

```
GITHUB_MCP_TOKEN=ghp_XXXXXXXXXXXXXXXXXXXXXXXXXXXX
```

This environment variable is used by the MCP GitHub Server to authenticate API requests and perform repository operations securely.

5. File Structure

The project follows a clean two-tier directory structure separating backend (Python/FastAPI) from frontend (Next.js):

```
zero-touch-vulnerability-remediation/
├── backend/                      # Python backend (FastAPI + LangGraph)
│   ├── agents/                  # Multi-agent logic (Parsing, Intel, Remediation,
etc.)                             etc.)
│   ├── lib/                     # Database migrations and utilities
│   ├── main.py                  # FastAPI entry point
│   ├── app.py                   # Application routing and setup
│   └── requirements.txt          # Python dependencies
├── frontend/                    # Next.js 15 Web Application
│   ├── app/                     # Next.js App Router (pages & layouts)
│   ├── components/              # Reusable React/Tailwind UI components
│   └── package.json              # Node dependencies
├── test/                        # Sample payloads for testing
│   ├── scanner1_trivy_filtered.json
│   └── scanner2_grype_filtered.json
└── flow.png                     # System Architecture Diagram
```

6. Components

The Zero Touch Vulnerability Remediation System leverages cloud services, automation tools, and AI technologies to automate vulnerability detection, remediation, validation, and tracking.

Component	Purpose & Role in Project	Key Functionalities
Amazon EC2	Serves as the remediation execution environment where AI-generated scripts, validation checks, and security scans are executed.	Script execution, validation, scanner hosting, log generation
Amazon S3	Centralized storage layer used for remediation logs, validation reports, workflow artifacts, and compliance evidence.	Log storage, report storage, artifact management
AWS Systems Manager (SSM)	Provides secure remote command execution on EC2 instances without requiring SSH access. Acts as the execution bridge between automation workflows and remediation infrastructure.	Remote execution, instance management, workflow automation
Jira	Manages the complete vulnerability lifecycle from detection to resolution while supporting Human-in-the-Loop approval workflows.	Ticket creation, status tracking, approvals, audit trail
GitHub Actions	Automates workflow execution and orchestrates communication between GitHub repositories and AWS services.	CI/CD automation, AWS integration, workflow triggering
Model Context Protocol (MCP)	Enables AI agents to interact with external tools and enterprise systems through a standardized interface.	Tool integration, context sharing, action execution
Tavily Search API	Provides external threat intelligence and remediation references to enrich AI-driven decision making.	Web search, CVE research, remediation guidance
OpenAI API	Powers vulnerability analysis, remediation generation, root-cause identification, and reasoning tasks within the platform.	Vulnerability analysis, remediation generation, log analysis
Gemini API	Acts as a fallback source for vulnerability intelligence when NVD or other primary sources provide incomplete information.	Vulnerability enrichment, CVE research, web-assisted information retrieval

7. Technology Stack

The Zero Touch Vulnerability Remediation System is built using a modern technology stack that enables AI-driven automation, real-time communication, workflow orchestration, and efficient data management.

Technology	Purpose
Next.js	Used to build the user dashboard and monitoring interface. It provides a fast, responsive UI for viewing vulnerabilities, remediation progress.
LangChain	Integrates Large Language Models (LLMs) with external tools and APIs. It enables AI-powered reasoning, tool calling, and automation of vulnerability remediation tasks.
LangGraph	Manages agentic workflows using graph-based execution. It handles workflow state, multi-step decision making, and smooth execution of remediation processes.
FastAPI	Acts as the backend framework for handling API requests and connecting different system components. It ensures fast communication between the frontend, AI agents, and databases.
Python	Primary programming language used to implement automation, vulnerability analysis, AI workflows, and remediation logic. It provides flexibility for integrating multiple security tools.
WebSocket	Enables real-time communication between the backend and frontend. It allows users to receive live updates on vulnerability status and remediation progress without refreshing the dashboard.
PostgreSQL	Stores vulnerability records, remediation history, asset information, and other system data. It provides secure, scalable, and reliable data management.
SQLite	Used as a lightweight database for storing workflow states and temporary agent data. It supports fast local storage and efficient workflow execution.

This technology stack provides scalability, real-time processing, AI-powered decision-making, and efficient vulnerability remediation management.

8. Understanding the Pipeline

The platform executes vulnerability remediation through a sequence of specialized AI agents, each responsible for a distinct stage of the pipeline. The following sections describe each agent's purpose, working principle, and integration within the overall workflow.

8.1 Parser Agent

Overview

The Parser Agent is responsible for converting heterogeneous vulnerability scanner outputs into a standardized tabular format that can be consumed by downstream remediation agents.

Since different security scanners generate reports in different JSON structures, a static parser would require separate parsing logic for every scanner. To overcome this limitation, an AI-driven adaptive parsing mechanism was developed.

Working Principle

The Parser Agent follows a multi-stage workflow:

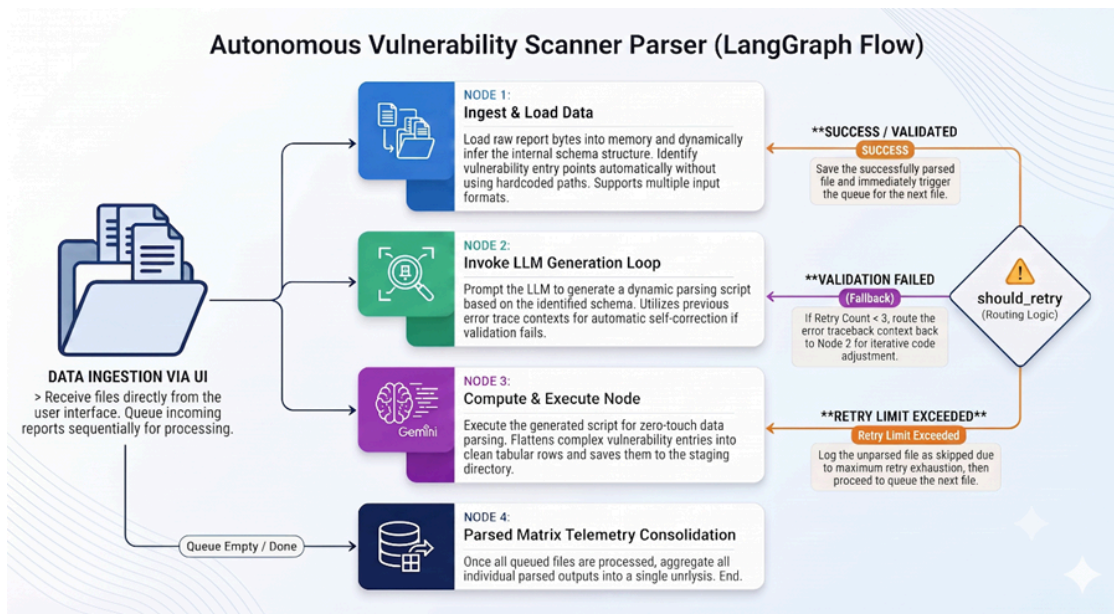


Figure 1.2: High-Level System Architecture

- Discover scanner output files.
- Load raw JSON reports.
- Analyze the report structure and identify vulnerability entry points.

- Generate scanner-specific parsing logic using a Large Language Model (LLM).
- Execute the generated parser.
- Validate the extracted data.
- Retry parser generation automatically if validation fails.
- Export normalized vulnerability data into CSV format.

This approach enables the system to support multiple scanners without requiring manually written parsers for each tool.

Input

The Parser Agent accepts raw vulnerability reports generated by security scanners such as: Trivy, Snyk, Gype, OWASP, ZAP, and other industry supported scanner outputs.

Output

The output is a normalized CSV dataset containing vulnerability information extracted from scanner reports.

Example Fields: CVE ID, Severity, Package Name, Installed Version, Fixed Version, CVSS Score, Description

Self-Healing Capability

One of the key capabilities of the Parser Agent is its self-correction loop.

If the generated parsing logic produces empty results or encounters runtime errors:

- The error is captured.
- The failure context is provided back to the LLM.
- A new parser is generated.
- Validation is performed again.

This cycle continues until successful extraction is achieved or the retry threshold is reached.

8.2 Normalization Agent

Overview

The Normalization Agent is responsible for transforming heterogeneous vulnerability data into a unified enterprise-wide schema.

Although the Parser Agent successfully extracts vulnerability information from different security scanners, each scanner still produces different column names, formats, severity labels, and metadata structures. These inconsistencies make downstream processing difficult.

To address this challenge, the Normalization Agent standardizes all parsed outputs into a common schema that can be consumed consistently by prioritization, remediation, validation, and reporting components.

Working Principle

The Normalization Agent follows an automated normalization workflow:

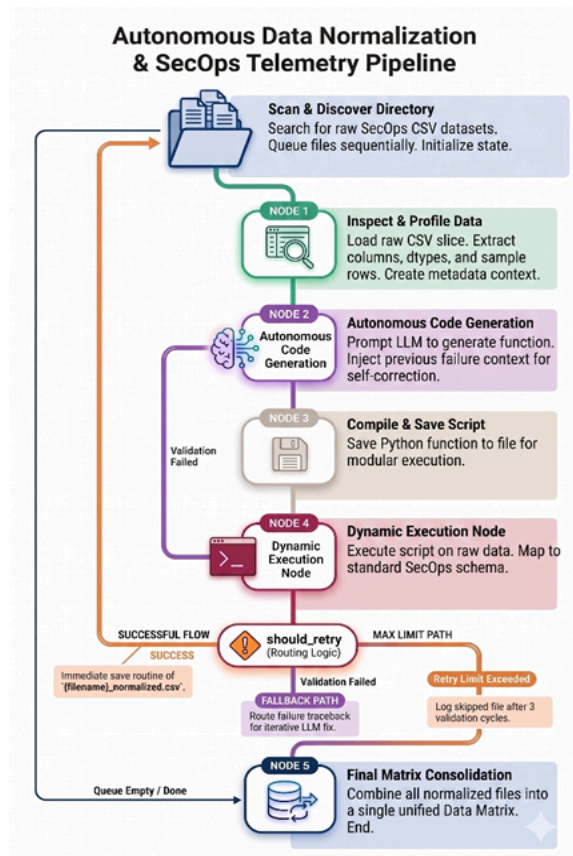


Figure 1.3: Vulnerability Enrichment Agent – Part 2 Flow Diagram

1. Discover parsed CSV files generated by the Parser Agent.
2. Analyze column names, data types, and sample records.
3. Generate normalization logic dynamically using an LLM.
4. Transform scanner-specific fields into a standardized schema.
5. Validate the normalized output.
6. Retry normalization automatically if validation fails.
7. Export normalized datasets.

8. Merge all normalized datasets into a unified working dataset.

This enables vulnerability information from multiple scanners to be represented in a consistent format regardless of the original source.

Standardized Schema – Core Fields

Field	Description
vuln_id	CVE ID or scanner-specific identifier
severity_raw	Original vulnerability severity
ip	Affected IP address
hostname	Target hostname
port	Affected port
instance_id	Cloud instance identifier
component	Vulnerable software/package
affected_version	Installed vulnerable version
cvss_score	CVSS severity score
cvss_vector	CVSS vector string
description	Vulnerability description
references	Related references and advisories

Input & Output

Input: Parsed CSV files generated by the Parser Agent.

Output: Standardized normalized CSV files and a normalized dataset for downstream prioritization and remediation.

Self-Healing Capability

Similar to the Parser Agent, the Normalization Agent includes a validation and retry mechanism.

If the generated normalization logic:

- Produces invalid output
- Returns an empty dataset
- Generates runtime errors

the failure context is automatically provided back to the LLM, which regenerates improved normalization logic and retries execution.

This process continues until successful normalization is achieved or the retry threshold is reached.

8.3 Vulnerability Enrichment Agent – Part 1

Core Objective

The Vulnerability Agent acts as the foundational data ingestion and sanitization layer. Its primary role is to ensure that every alert generated by upstream security scanners is normalized into a globally recognized standard (CVE) and enriched with authoritative NIST metrics.

Execution Workflow (Node-by-Node)

1. [Node: Read CSV & Check CVE] Data Ingestion & Masking: The agent securely loads the scanner output. It immediately applies a mathematical mask to identify missing, blank, or proprietary vulnerability IDs that do not match the standard CVE- format.
2. [Node: Gemini LLM Fallback] Intelligent Data Recovery: Instead of discarding alerts with missing IDs, the agent routes them to a Gemini LLM. The LLM acts as an analyst, reading the vulnerability's title and description to accurately infer the correct CVE ID. Unrecoverable alerts are dropped and saved to a `rejected_records` file.
3. [Node: Query DB Cache] Database-First Retrieval: To process records instantaneously, the agent queries a local PostgreSQL database. (This database is kept constantly updated by a backend cron job that syncs with national databases). The agent fetches all important base metrics directly from this local database, including the CVSS Score, CVSS Vector, Fix Version, and Published Date.
4. [Node: Fetch Missing via NVD API] Targeted API Fallback: If a vulnerability is not found in the local database cache, the agent dynamically batches those missing CVEs. It executes a concurrent HTTP request against the NVD API specifically to fetch the missing metrics.
5. [Node: Save CSV & Batch Upsert] Data Sync: The agent merges the database results and API results, writes them to the CSV, and performs an asynchronous batch upsert to save the newly fetched records back into the PostgreSQL database for future runs.

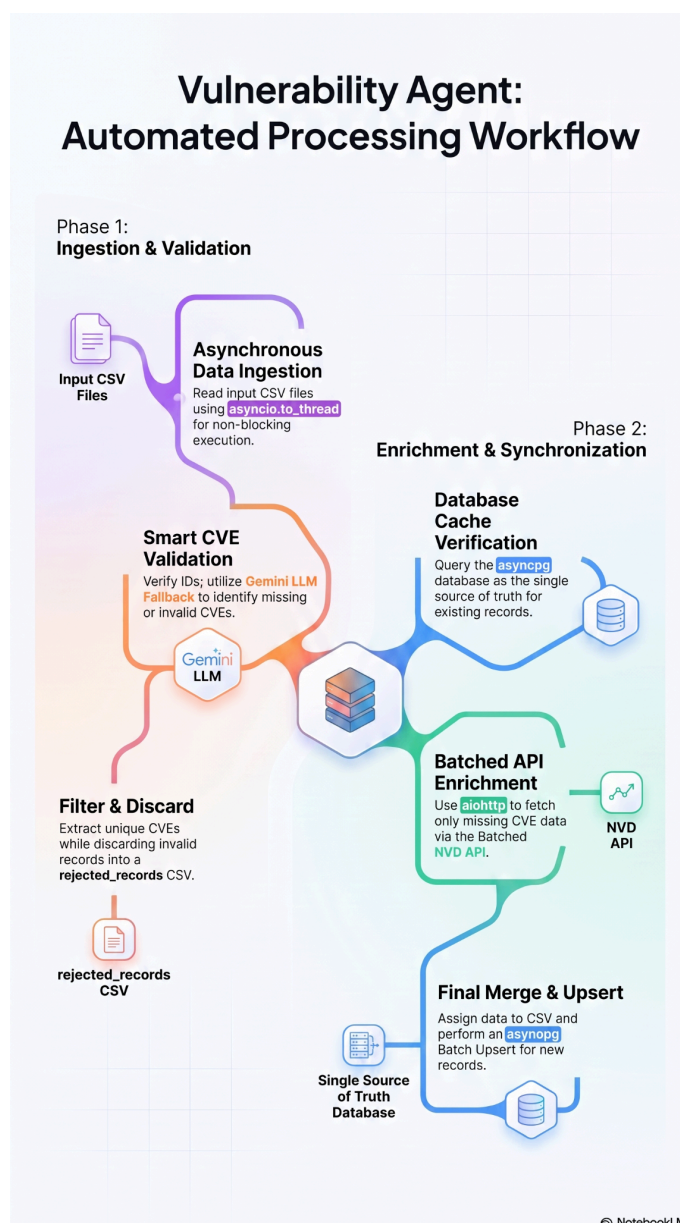


Figure 1.4: Vulnerability Enrichment Agent – Part 1 Flow Diagram

8.4 Vulnerability Enrichment Agent – Part 2

Core Objective

The Vuln Intel Agent acts as the threat intelligence engine. It takes the normalized vulnerabilities and correlates them against global, real-world attack metrics to determine if hackers are actively weaponizing them in the wild.

Execution Workflow (Node-by-Node)

1. [Node: Nightly Intel Cron Sync] The Global Threat Mirror: Operating independently in the background, a cron script runs nightly to download over 250,000 records of global cyber intelligence (the entire CISA KEV catalog, FIRST EPSS database, and Exploit-DB repositories) and caches them in the local PostgreSQL database.
2. [Node: Query DB Cache] Instantaneous Threat Correlation: During runtime, the Intel Agent reads the enriched CSV and queries the local intelligence mirror. It instantly extracts the exact EPSS Score, KEV Flag (0 or 1), and Exploit Counts for your specific vulnerabilities.
3. [Node: Fetch Missing via API] API Failover Mechanics: If any vulnerabilities are missing from the local intelligence database, the agent executes a targeted batched request to the FIRST.org API to fetch the missing EPSS scores, and concurrently pulls the CISA KEV data to fill the gaps.
4. [Node: Merge & Assign] Evidence-Based Aggregation: The agent mathematically weighs the exploit evidence, ensuring that if a vulnerability is flagged on the CISA KEV list, it receives the highest priority exploit flags.
5. [Node: Save CSV & Batch Upsert] Data Sync: The finalized threat intelligence metrics are written back to the CSV, and any newly fetched API records are asynchronously batch-upserted back into the database cache.

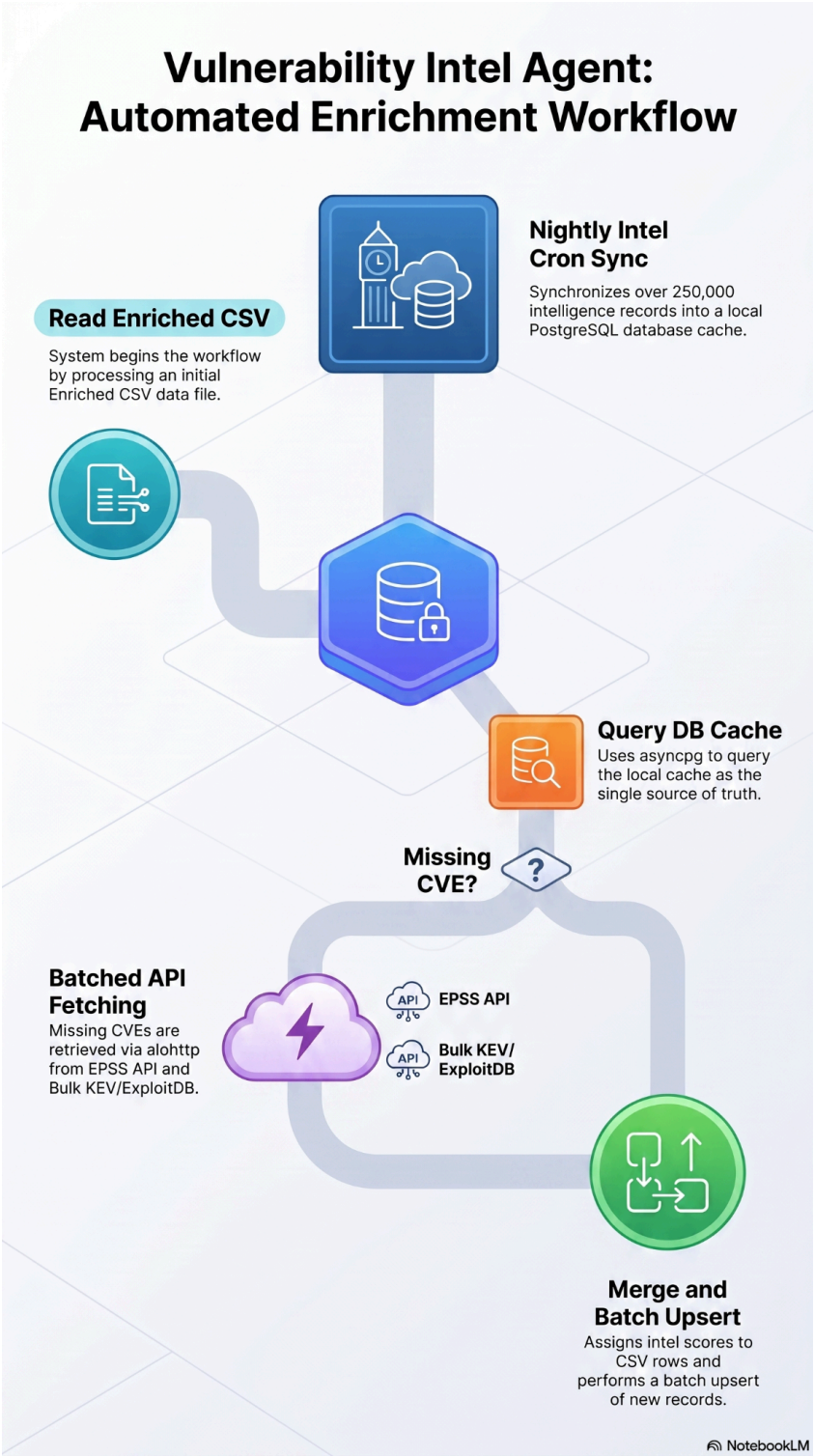


Figure 1.5: Detailed Architecture Diagram

8.5 Asset Enrichment Agent – Part 1

Overview: Why Two Separate Agents

These two agents together handle everything from receiving a raw normalized CSV to having every asset fully scored and written back to the database. They are split into separate files deliberately — not for convenience, but because they have genuinely separate concerns.

Agent 1 — Asset Lookup	Agent 2 — Asset Criticality
Data resolution layer	Scoring layer
Resolves asset identity (instance_id, hostname, IP)	Receives enriched CSV from Agent 1
Pulls existing DB records or generates new UUIDs	Skips assets already scored (no recomputation)
Outputs fully enriched CSV for Agent 2	Runs full inference pipeline on new assets
Does not compute criticality scores	Upserts final scores back to DB

Input & Output

Agent 1 reads `working.csv` — the output of the normalization stage — which contains one row per vulnerability finding with asset identity fields (`instance_id`, `hostname`, `ip`). By the time Agent 1 finishes, every row has an `asset_id` and rows belonging to existing assets are enriched with all known metadata. The enriched CSV is saved back to the same path for Agent 2.

Pipeline Structure

Agent 1 is a LangGraph graph with six nodes in sequence. The reason for a graph instead of a flat script: state management and error isolation. Each node receives the current state, does its work, and returns an updated state. If any node fails, a routing function checks the status field and short-circuits to END immediately.

Node Sequence: Load CSV → DB Lookup & Split (existing vs. new) → Enrich Existing Assets → Assign UUIDs & Insert Skeletons → Update last_seen Timestamps → Save CSV → Hand off to Agent 2

Node 1 — Load & Validate CSV

The CSV is read and columns are normalized to lowercase. Rows missing all three identity fields (`instance_id`, `hostname`, `ip`) are rejected immediately — they cannot be matched to any asset.

Rejected rows are written to `rejected_asset_records.csv` for visibility. Keeping them out early prevents orphan records from propagating through the pipeline.

DB Lookup — Priority-Ordered Matching

Assets are identified in three ways, used in strict priority order:

- 1. `instance_id` first — immutable cloud identifier
- 2. `hostname` second — more stable than IP
- 3. IP address last — can change (DHCP, cloud re-provisioning)

To minimize round trips, all three lookups are bulk queries. The agent collects all unique values from the CSV, fires one query per field type, and builds three in-memory maps. Each row is then resolved against those maps using the priority order — no per-row DB calls.

Enriching Existing Assets

For existing assets, the DB record already contains computed fields: `inferred_role`, `exposure_score`, `environment`, `cloud_type`, dependency scores, and criticality score. These are written directly into the CSV rows so Agent 2 skips them. A match key of (`instance_id`, `hostname`, `ip`) pairs CSV rows to DB records — handling cases where the same asset appears in multiple rows. The `last_seen` timestamp on each matched row is also updated to the current time.

New Assets — UUID Generation & Skeleton Insert

For assets not in the DB, a UUID is generated in-process. However, Agent 1 does not trust the locally generated UUID as final — because `ON CONFLICT DO NOTHING` on insert means a concurrent process could have inserted the same asset between lookup and insert. The agent inserts all new skeletons in a single batch, then immediately re-queries the DB for the actual `asset_id` using each asset's identity fields. Whatever the DB returns is treated as authoritative.

SKELETON FIELDS ONLY: `asset_id`, `hostname`, `instance_id`, `ip_address`, `first_seen`, `last_seen` — full profile (roles, scores, criticality) is filled in by Agent 2. This keeps each agent's write scope clean.

Batch Operations Throughout

Every DB interaction in Agent 1 is batched — no per-row queries at any stage:

- Three lookups (instance_id, hostname, IP)
- Skeleton inserts — execute_batch in a single round trip
- last_seen updates — single batched update across all asset_ids

Retry Logic & Transient Error Handling

All DB connections are wrapped in a retry helper: catches transient errors (connection failures, operational errors, timeouts) and retries up to 3 times with a 5-second wait. Non-transient errors (bad SQL, constraint violations) are not retried — they surface immediately.

Why Everything Is Async

The agent is I/O-heavy: reading CSVs, opening DB connections, executing queries, writing files. Async keeps the process non-blocking. Blocking operations (file I/O, psycopg2 queries) are offloaded to a thread pool via `asyncio.to_thread`. But computation of all assets is concurrent.

NOTE: Despite being async, the pipeline is sequential by design. Each node waits for the previous to complete — async here means non-blocking, not parallel.

Control Transfer to Agent 2

After saving the enriched CSV, Agent 1's final node calls Agent 2's compiled LangGraph graph directly via `invoke`, passing the working CSV path and a fresh initial state. Agent 1's responsibility ends here. If Agent 2 fails, the error is captured and bubbled up through Agent 1's state.

8.6 Asset Enrichment Agent – Part 2

Input & Output

Agent 2 receives the enriched `working.csv` from Agent 1. Its job: compute criticality scores for assets that are new — those whose criticality fields were not populated by Agent 1 because they had no existing DB record. By the end, every row has a complete criticality profile, all assets are upserted to DB, and the CSV is written back.

What Gets Skipped & Why

Agent 2 immediately identifies assets that already have all required criticality fields populated (copied from DB by Agent 1) and skips them entirely. No recomputation on unchanged assets — this avoids unnecessary LLM calls.

SKIP LOGIC: A row is skipped if none of the criticality columns are null. The check is done on unique `asset_ids`, not raw rows — so an asset appearing in 50 vulnerability rows is evaluated once, not 50 times.

Inference Pipeline — Deterministic First

For each new asset, five inference steps run in sequence:

Step	What It Does
Role Inference	Priority chain: open port → port-to-role map, then component keywords → component-to-role map, then hostname hints. Covers databases, auth systems, web servers, API backends, CI/CD platforms, management/bastion hosts.
Exposure Inference	Public IP → 1.0. Private IP running internet-exposed service (web, SSH, K8s API, Elasticsearch) → 0.8 (private but tunnel exposed). Standard private IP → 0.5. Localhost → 0.2. Falls back to 0.5.
Environment Inference	Scans hostname, component, and tags for production vs. non-production keywords.
Cloud Detection	Hostname checked against patterns for AWS, Azure, GCP managed services. Affects blast-radius in dependency scoring.
Dependency Scoring	Starts from a role-based baseline (AUTH/DATABASE → CRITICAL, ENDPOINT → LOW). Modifiers: internet-facing, production, high-value cloud service type, or production CI/CD raises score. Non-prod or isolated/localhost lowers it.
Final Formula	Asset's Criticality Score = Exposure Score (0.50) * Dependency Score (0.50)

CRITICALITY SCORE: This single number represents the asset's overall risk weight in the prioritization engine.

LLM Fallback — Only When Deterministic Inference Hits a Wall

The LLM is invoked only if ALL FOUR conditions are true simultaneously:

- Role is UNKNOWN
- Environment is UNKNOWN
- Exposure resolved to default unknown value (0.5, no signal)
- Cloud type is NONE

If any of these fields was resolved deterministically, the LLM is not called. LLM calls add latency and cost; calling them where a partial signal already exists would add noise.

LLM: Google Gemini 2.5 Flash, invoked async. After each LLM call, dependency and criticality scores are recomputed from scratch using the resolved values.

Concurrency Within the Node

All new assets are processed concurrently using `asyncio.gather` — if there are 20 new assets, their inference pipelines all start simultaneously. Deterministic inference is pure computation, entirely dependent on CPU/System performance, with zero latency overhead. This makes the agent fast on large batches without overwhelming external services.

DB Upsert — One Round Trip for All Assets

All assets are written back to DB in a single batch upsert using `ON CONFLICT (asset_id) DO UPDATE`. Key behaviour:

- Existing records: updated with new computed fields
- New records: inserted
- `first_seen`: preserved using `COALESCE` — never overwritten
- `last_seen`: always set to current timestamp
- Deduplication before upsert: each `asset_id` included only once, regardless of how many vulnerability rows it appears in.

Writing Back to CSV

After the DB upsert, the final node writes the fully scored list to CSV. This is the handoff artifact to the next prioritization agent. Every row now has a valid `asset_id`, a full criticality profile, and enriched asset metadata. Rows belonging to existing assets carry their historical criticality scores, unchanged.

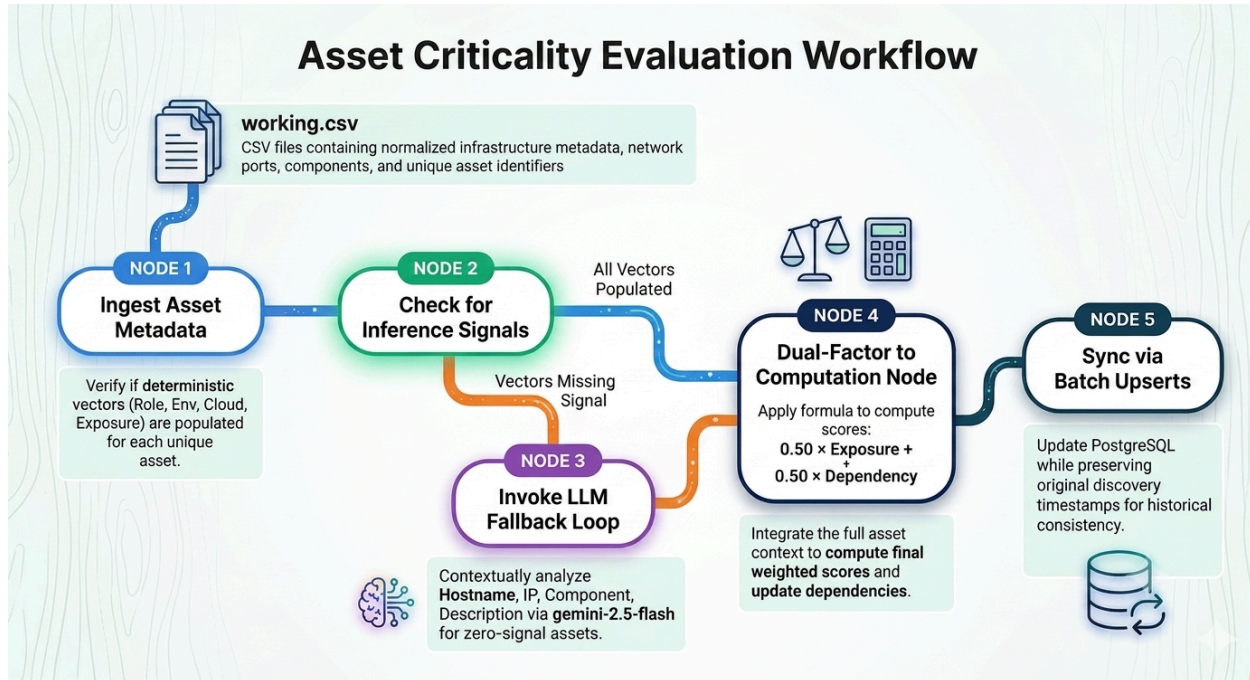


Figure 1.6: Asset Criticality Agent Architecture

8.7 Asset Criticality Agent

The Asset Criticality Agent is integrated within the Asset Enrichment Agent Part 2. It computes the criticality score for each asset using the deterministic inference pipeline described above (Role → Exposure → Environment → Cloud → Dependency → Final Formula). This score becomes the foundational input for the Prioritization Agent, ensuring business context drives remediation priorities rather than raw technical severity alone.

8.8 Prioritization Agent

Core Objective

The Prioritization Agent acts as the mathematical brain of the pipeline. It abandons theoretical severity and blends technical data with real-world threat intelligence and business context to output a single, highly actionable Priority Score.

Execution Workflow (Node-by-Node)

- [Node: Query DB for Existing Scores] Historical Caching: The agent queries the `asset_vulnerabilities` database table. If an exact (Asset ID + Vuln ID) pair has already been prioritized in a previous run, the score is instantly retrieved to save processing time.

- [Node: Extract Normalization Factors] Contextual Aggregation: For new calculations, the agent ingests the fully enriched dataset. This includes technical severity (CVSS), threat intelligence (EPSS/KEV), and operational data (Fix Availability).
- [Node: Calculate Custom Priority Formula] The Business-Context Risk Engine: The agent normalizes all data points onto a strict 0.0 to 1.0 scale and feeds them into a custom, heavily weighted algorithm.

Priority Formula Components:

- Asset Criticality (25%): The core business context. This dictates the actual impact on the company if the specific vulnerable server goes down.
- CVSS (25%): Foundational technical severity.
- EPSS & KEV (40%): Active exploitation likelihood in the wild.
- Fix Availability & Vulnerability Age (10%): Operational metrics indicating how actionable and old the threat is.
-
- [Node: Map to Industry Level] Actionable Output: The algorithm generates a precise 0.0 to 10.0 score, which is instantly mapped to an industry-standard tier (LOW, MEDIUM, HIGH, CRITICAL).
- [Node: Save CSV & Batch Upsert] Handoff to Remediation: The final, context-aware intelligence is exported and saved to the database. This output seamlessly bridges the pipeline to the downstream Remediation Agent for automated patching, while simultaneously syncing with Jira for strict ticket tracking and lifecycle management.

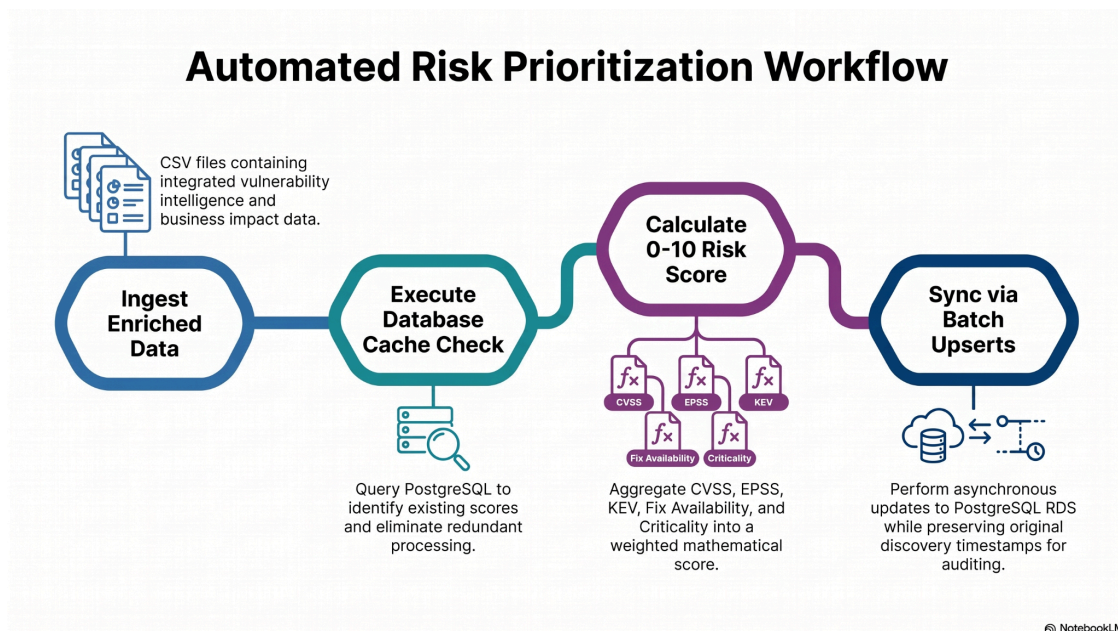


Figure 1.7: Full Pipeline Architecture Overview

8.9 Remediation Agent

The Remediation Workflow is responsible for automatically generating, validating, and managing security patches for detected vulnerabilities. Starting from a prioritized vulnerability, the workflow performs vulnerability research, generates remediation scripts using an LLM, validates the generated solution in an isolated environment, and finally deploys the approved remediation to the target system. Throughout the process, the workflow integrates with GitHub, JIRA, AWS services, and a lightweight database to track progress, support human review, and recover safely from interruptions.

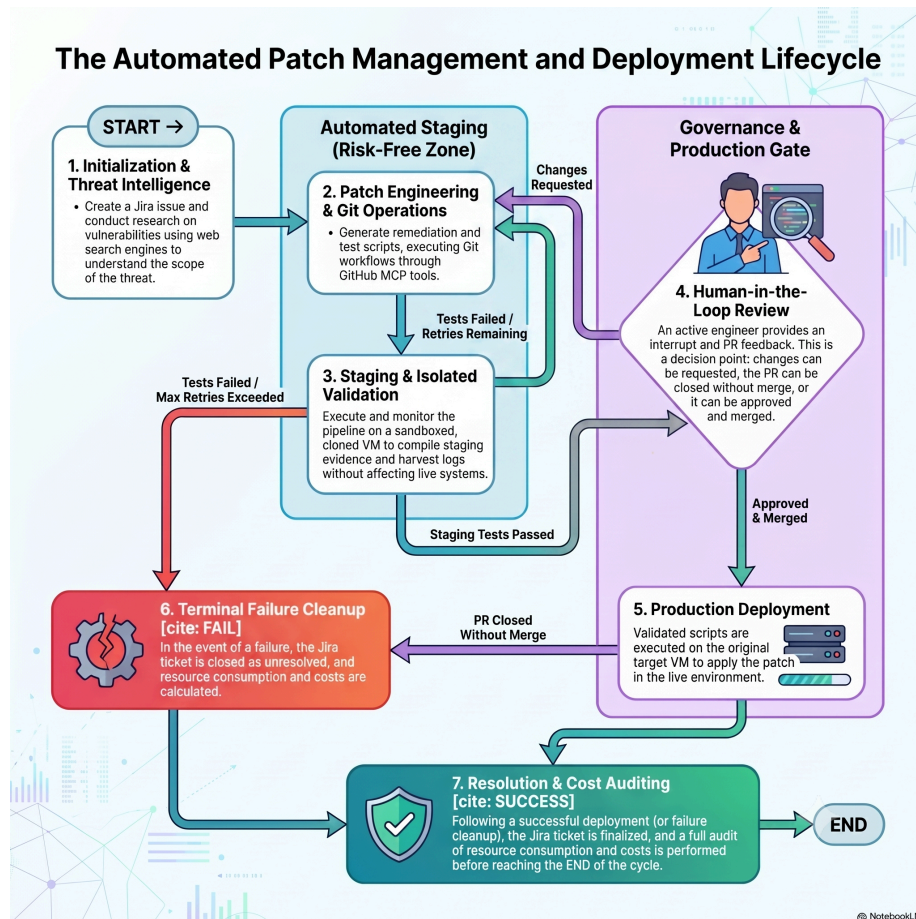


Figure 1.8: Remediation Agent Overall Workflow

1. Workflow Initialization

Once a vulnerability is selected from the remediation queue, the FastAPI backend creates a unique workflow (thread) for that vulnerability and records its initial status in the database. This enables the system to track the execution of every remediation task independently. At the same time, a JIRA ticket is created to log the vulnerability and track the remediation process from start to finish.

2. Vulnerability Research

The workflow first performs automated research on the vulnerability using web search tools. It gathers information from trusted security advisories, vendor documentation, CVE references, and patch notes. This additional context helps the LLM understand the root cause of the vulnerability, possible fixes, affected software versions, and recommended remediation approaches.

3. Script Generation

Using both the original vulnerability information and the collected research, the LLM generates three Bash scripts:

- Precheck Script – Collects the current system state before any changes are made.
- Remediation Script – Applies the security patch or configuration change.
- Validation Script – Confirms that the vulnerability has been successfully remediated.

The agent also generates a GitHub branch name and a short remediation summary. These scripts are stored together in a single `scripts.json` file, which serves as the input for the CI/CD pipeline.

4. GitHub Automation

The workflow then uses GitHub MCP tools to automate all GitHub operations. A remediation branch is created (if required), the generated `scripts.json` file is committed, and a Pull Request (PR) is opened automatically.

Once the Pull Request is created, the PR number and the workflow thread ID are stored in the `'pr_mappings'` table of the `'state_db.sqlite'` database. This mapping allows future GitHub webhook events (such as review approvals, review comments, or PR merges) to identify exactly which remediation workflow should be resumed. The workflow status is also updated to indicate that remediation is currently in progress.

5. Validation in a Sandboxed Environment

Every time the Pull Request is created or updated, the CI/CD pipeline is triggered. The pipeline extracts the generated scripts from `scripts.json` and executes them inside a sandboxed environment (a cloned virtual machine) rather than on the original production system.

The scripts are executed in the following order:

Precheck → Remediation → Validation

Execution logs from every stage are automatically stored in Amazon S3.

If any stage fails, the workflow retrieves the execution logs from S3 and provides them to the LLM as feedback. Using this information, the agent generates an improved remediation, updates the Pull Request with the new scripts, and triggers another validation cycle. This self-healing loop continues until the remediation succeeds or the retry limit is reached.

6. Human-in-the-Loop (HITL) Review

After the remediation successfully passes validation in the sandboxed environment, the workflow updates the JIRA ticket with the latest progress and pauses for human review. The workflow status is updated to 'WAITING_FOR_HUMAN_APPROVAL' in the 'workflow_state' table of the 'state_db.sqlite' database, allowing the agent to safely stop execution while preserving its current state & move to next vuln remediation while waiting for approval of previous ones.

A security engineer then reviews the generated remediation, validation results, and Pull Request.

- If changes are requested, the workflow automatically retrieves the reviewer comments through GitHub webhooks, resumes the paused workflow using the stored PR-to-thread mapping, regenerates the scripts based on the feedback, and starts another validation cycle.
- If the Pull Request is approved and merged, the validated scripts are executed on the original target virtual machine, ensuring that only human-approved remediations are applied to production systems.
- If the Pull Request is closed without approval, the workflow is terminated and the associated JIRA ticket is marked as unresolved.

7. Workflow Completion

Once the remediation is successfully applied to the target system, the workflow records the execution time, token usage and estimated LLM cost. The workflow state is updated to 'COMPLETED' for the current thread in the 'workflow_state' table of 'state_db.sqlite' database, and the corresponding JIRA ticket is marked as Resolved.

If the workflow encounters an unrecoverable error or fails after all retry attempts, its state is updated to 'FAILED' for the corresponding thread in the 'workflow_state' table. This ensures that every remediation workflow ends with a clear and traceable execution status.

9. Database Schema

Five tables. Each has a distinct responsibility — nothing overlaps by accident.

9.1 vulnerabilities

Stores static CVE data. Once a CVE is published, its title, description, CVSS score, and fix version rarely change. This table is the canonical reference for what a vulnerability is — not how urgent it is in context.

Key fields:

- `vuln_id` (varchar, PK) — CVE identifier
- `title / description` (varchar / text) — what the vuln is
- `cvss_score / cvss_vector` (float / varchar) — base severity from NVD
- `fix_version` (varchar) — version that patches it
- `published_date` (date) — when the CVE went public

9.2 vulnerability_intel

Stores live intel on top of a CVE — EPSS score, KEV flag, exploit count. These shift daily as new exploits drop or CISA updates the KEV list. Kept separate so it can be refreshed on a daily schedule without touching the static vuln record.

Key fields:

- `vuln_id` (varchar, FK → vulnerabilities) — links back to the CVE
- `epss_score` (float) — probability of exploitation in the wild (0–1)
- `kev_flag` (float) — whether CISA flagged this as a known exploited vuln
- `exploit_exists / exploit_count` (int / int) — are there public exploits and how many
- `last_updated` (timestamp) — when this intel was last refreshed

9.3 assets

Stores asset identity and the dependency context inferred by the pipeline. The criticality and dependency fields carry the business-context answer to 'how important is this machine?' — which is what drives prioritization downstream.

Key fields:

- `asset_id` (uuid, PK)
- `ip_address` / `hostname` / `instance_id` (varchar) — identity fields used for matching
- `inferred_role` (varchar) — what kind of service this asset runs (e.g. DATABASE, AUTH)
- `exposure_score` (float) — network accessibility — 0.2 (localhost) to 1.0 (public)
- `environment` (varchar) — prod vs non-prod
- `cloud_type` (varchar) — managed cloud service or self-hosted
- `dependency_level` / `dependency_score` (varchar / float) — blast-radius weight — how critical to the system
- `asset_criticality_score` (float) — final combined score used in prioritization
- `first_seen` / `last_seen` (timestamp) — scan lifecycle tracking

9.4 asset_vulnerabilities

Junction table between assets and vulnerabilities — but it does more than link them. For every `asset↔vuln` pair, it holds the computed priority: how urgent is this specific CVE on this specific asset. The same CVE on a public-facing auth server vs. an isolated dev box gets very different priority scores. That context lives here.

Key fields:

- `asset_id` + `vuln_id` (composite PK)
- `severity_raw` / `severity_score` (varchar / float) — scanner-reported severity
- `component` / `affected_version` / `port` (varchar / varchar / int) — where exactly on the asset the vuln exists
- `fix_available` (int) — whether a patch exists
- `priority_score` / `priority_level` (float / varchar) — final computed priority for this `asset↔vuln` pair
- `first_seen` / `last_seen` (timestamp) — when this finding was first and last observed

9.5 vulnerabilities_history

Stores the execution history of remediation workflows. Each record represents a single remediation attempt for a specific asset and vulnerability, capturing execution outcome, resource usage, and performance metrics. This table provides auditability, traceability, and operational insights into the remediation pipeline.

Key fields:

- `thread_id` (varchar, PK) — unique identifier for the remediation workflow execution
- `asset_id` (varchar) — asset on which remediation was attempted
- `cve_id` (varchar) — vulnerability being remediated
- `score` (float) — risk/priority score associated with the vulnerability during remediation
- `resolved` (boolean) — indicates whether remediation was successfully completed
- `cost` (float) — estimated execution or LLM processing cost incurred
- `token` (int) — total tokens consumed during the remediation workflow
- `start_time` / `end_time` (timestamp) — timestamps marking the start and completion of the remediation process
- `time_taken` (interval) — total duration of the remediation workflow

Why it matters: Offloading historical data to PSQl ensures the active LangGraph/SQLite environment remains incredibly fast and free of database bloat, while still providing robust, relational data to the executive Dashboard. All dashboard widgets query this exact PSQl table.

9.6 Dataset Description

Due to the unavailability of production enterprise asset inventories, the platform utilizes a synthetically generated asset dataset designed to realistically simulate enterprise environments. The generated asset data includes attributes such as hostnames, IP addresses, instance identifiers, asset roles, environments, cloud providers, and business-criticality indicators. This approach enables the evaluation of asset contextualization and prioritization workflows without exposing sensitive organizational information.

For vulnerability intelligence, the platform leverages real-world vulnerability data sourced from the NVD, FIRST(EPSS), CISA(KEV), exploitdb etc. Vulnerability records, including CVE identifiers, CVSS metrics, descriptions, references, and associated threat intelligence, are periodically synchronized and maintained within a local database. Using authentic vulnerability data ensures that the prioritization, enrichment, and remediation workflows operate on realistic threat scenarios while preserving the confidentiality of enterprise asset information.

This combination of synthetic enterprise asset data and real-world vulnerability intelligence provides a practical and secure environment for developing, testing, and evaluating the zero-touch vulnerability remediation platform.

10. Cron Jobs

10.1 NVD Sync — Cron Job

Why This Exists

Scanners can surface tens of thousands of vulnerabilities in a single run. Making a live NVD API call for each one — to fetch CVSS score, severity, description — doesn't scale. At 1 lakh vulns it becomes a bottleneck, and NVD rate-limits unauthenticated callers to 5 requests per 30 seconds or even with API key its 50 requests per 30 seconds. The solution is a local vulnerabilities table that acts as a read-only mirror of NVD. All pipeline queries hit the local DB, not the API.

What It Does

Once a day, the job syncs any CVE data that NVD has added or modified since the last successful run — keeping the local table current without hammering the API on every scan.

- First-ever run: does a full historical import from 2002 to present, year by year.
- All subsequent runs: incremental only — fetches CVEs NVD modified since the last sync timestamp. If the job was down for 3 days, the next run automatically catches up the full gap.
- Upserts into the vulnerabilities table: title, description, cvss_score, cvss_vector, fix_version, published_date. Never overwrites a stored fix_version with a null — existing data is preserved.

Why Static Data Stays Local

Base CVSS score, severity, description — these don't change after a CVE is published. There's no reason to re-fetch them on every scan. Keeping them in a local DB means the pipeline can query millions of vuln records at DB speed, with no API dependency at scan time.

What Lives Here vs. vulnerability_intel

This job only maintains the static side of a CVE. Exploit status, EPSS probability, and KEV flag are live signals that shift daily — those are handled separately by the vulnerability_intel table and its own update process. The split keeps concerns clean: nvd_sync.py owns the stable reference data; intel updates own the dynamic risk signals.

10.2 Vulnerability Intelligence Cron Job

Why It Exists

When dealing with enterprise-scale security, your automated remediation pipeline must be blisteringly fast and highly resilient. Relying on external third-party APIs (like CISA or FIRST.org) during a real-time execution introduces massive network latency, strict rate limits, and a high risk of connection timeouts. This cron job exists to completely decouple the "heavy lifting" of downloading massive global datasets from the fast, real-time execution of the agents. It ensures that the pipeline never has to wait on the internet to make a critical remediation decision.

What It Does

Operating silently in the background on a 24-hour schedule, this script acts as a massive data aggregation engine:

- **Concurrent Global Downloads:** It uses cooperative multitasking (`asyncio.gather`) to simultaneously download over 250,000 records of cyber intelligence, including the FIRST.org EPSS catalog, the CISA KEV JSON feed, and Exploit-DB repositories.
- **Evidence Aggregation:** It parses these massive files in-memory, maps exploit counts to specific CVE IDs, and mathematically assigns the highest exploit evidence flags (e.g., locking the flag to 1 if the vulnerability appears on the CISA KEV list).
- **High-Speed Database Sync:** It packages this finalized intelligence and performs a massive, asynchronous batch-upsert (`ON CONFLICT DO UPDATE`) directly into the local PostgreSQL `vulnerability_intel` table, efficiently overwriting old data without duplicating records.

Why Intel Data Stays Local

- **Sub-Millisecond Speed:** By caching global intelligence locally, the Vuln Intel Agent can query thousands of vulnerabilities in milliseconds, rather than waiting minutes for network APIs to respond.
- **100% Uptime & Resiliency:** If government endpoints (like CISA) or organizational servers go offline for maintenance, your internal remediation pipeline is completely unaffected. It continues to run flawlessly using the local mirror.
- **Privacy & Operational Security (OPSEC):** If you query external APIs at runtime, you are essentially broadcasting your exact list of internal vulnerabilities to third-party servers. By bringing the entire global dataset in-house, your agents perform all their cross-referencing locally, ensuring your company's attack surface remains strictly confidential.

What Lives Here vs. The Vulnerability Agent Cron Job

To maintain pristine database architecture, the pipeline strictly separates technical data from threat context.

- The Vulnerability Agent Cron Job: This handles the Static Technical Baseline. It syncs with the National Vulnerability Database (NVD) to maintain a local repository of what a vulnerability is. It stores immutable facts: CVSS Base Scores, CVSS Vectors, textual descriptions, and official Patch Versions.
- The Vuln Intel Cron Job (This Script): This handles the Dynamic Threat Context. It stores highly volatile, rapidly changing data that dictates how hackers are behaving today. It stores EPSS exploit probabilities, CISA KEV active exploitation flags, and Exploit-DB proof-of-concept counts.

By separating these two cron jobs, the pipeline ensures that rapidly fluctuating hacker intelligence never bottlenecks or accidentally corrupts the foundational, static NIST vulnerability data.

11. Dashboard

The frontend dashboard serves as the operational command center, pulling analytics directly from the PostgreSQL historical records via the `/api/v1/dashboard` endpoint.

Note: Data is served exclusively from the `vulnerabilities_history` table in PostgreSQL, which acts as the historical `vulnerabilities_history` once migrated from the local SQLite queue.

11.1 Dashboard Metrics

Average Cost per Vulnerability

- Purpose: Tracks the financial efficiency of the LLM agents.

Average MTTR (Mean Time To Remediate)

- Purpose: Measures the speed of the autonomous pipeline.

Total Vulnerabilities

- Purpose: Tracks total volume processed by the system historically.

Total Solved Vulnerabilities

- Purpose: Validates the efficacy of the LLM patches.

Tokens Used & Historical Token Consumption

- Purpose: Monitors LLM API consumption and limits.

Success Rate

- Purpose: A high-level indicator of agent accuracy.

Vulnerabilities by Severity

- Purpose: Provides a risk profile breakdown of the historical queue.

Fleet Distribution / Recent Activity

- Purpose: An audit log of the most recently processed system tasks.

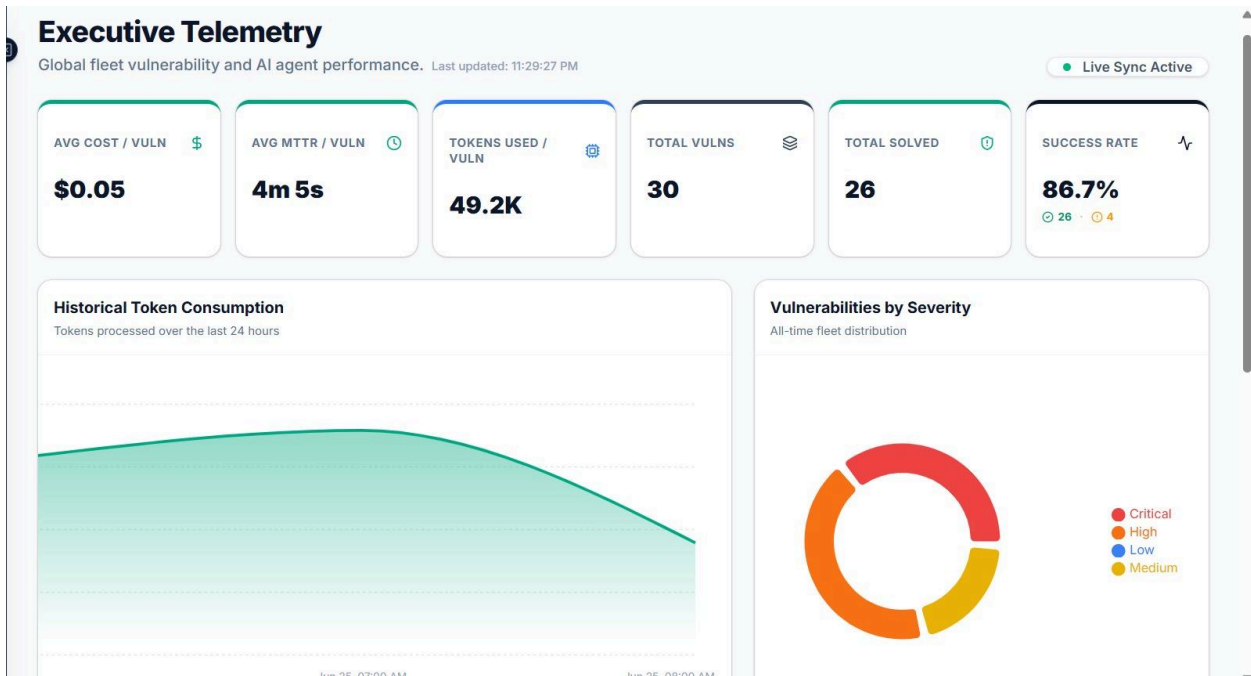


Figure 1.9: Dashboard Overview – Command Center

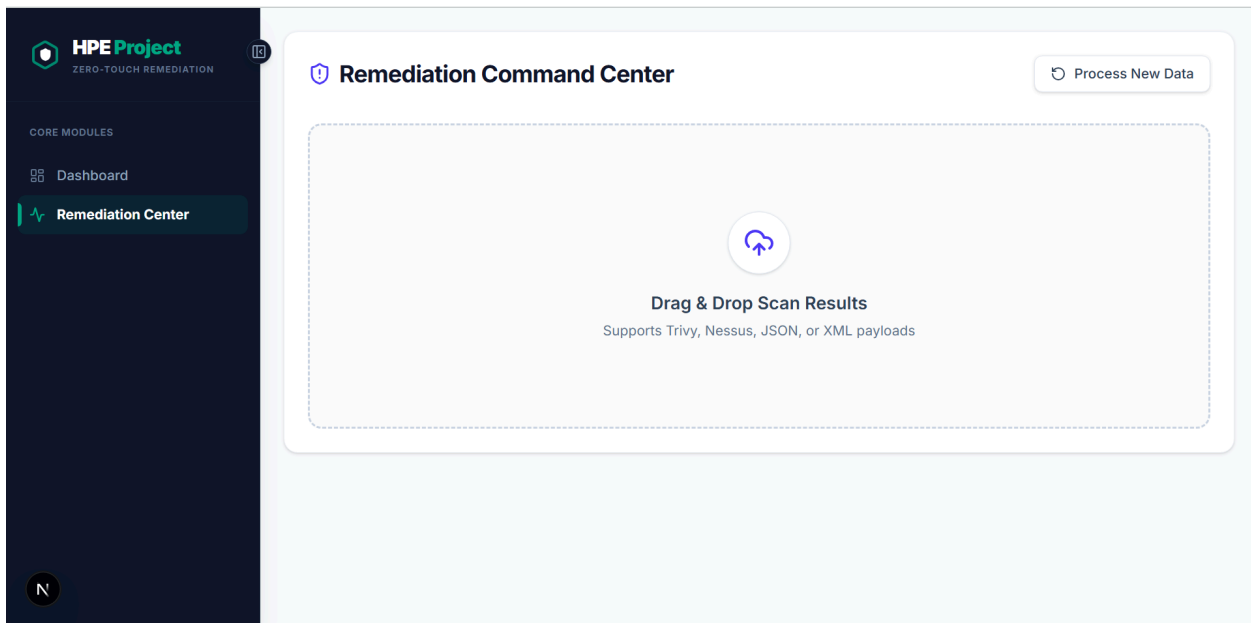


Figure 1.10: Dashboard Overview – Operational Command Center

12. Agent Performance Evaluation

For an autonomous, agent-driven system, a structured evaluation framework is absolutely critical to justify human trust and guarantee operational safety before the code hits production. The agent operates in a multi-stage pipeline: it first executes remediation scripts within a cloned sandbox environment and upon success, it pauses for human feedback. It deploys to the original VM only after explicit approval. Because a failure at any point in this chain can cause system drift or downtime, we must precisely quantify the agent's behavior and its engineering efficiency. This framework provides the essential metrics required to monitor the system's reliability.

12.1 Evaluation Objectives

- **Initial Precision & Code Accuracy:** We want to see how often the AI gets the fix right on its very first try. This measures how well the AI truly understands the security flaw from the start, without needing to rely on guessing, trial-and-error, or fixing its own mistakes.
- **Value for Money:** We track how much the AI costs on average to remediate each vulnerability, ensuring it delivers a good return on investment.
- **Speed:** We measure how fast the system works from start to finish, making sure to separate the time the AI actually spends running code from the time it sits around waiting for a human to approve its work.
- **Trust and Safety:** We look at how often human operators approve the AI's proposed fixes to make sure the team genuinely trusts its work and that it meets strict safety standards.

12.2 Categorized Key Performance Indicators (KPIs)

Core System Metrics

Metric Category	Metric Name	Definition	Why It Matters	Interpretation
Resolution Outcome	Resolution Success Rate (RSR)	The ratio of successfully closed vulnerabilities to total processed tickets.	Defines the core capability and reliability of the remediation engine.	High: High systemic trust. Low: Frequent failures, bad script generation.
Agent Execution	Mean Active Execution Time (MAET)	Average duration the agent actively computes, runs scripts, and parses logs, excluding human wait-time.	Measures processing velocity and operational efficiency of the agent architecture.	Low: Optimal performance. High: Infinite loops or complex script generation.
Script Reliability	First-Pass Success Rate (FPSR)	Percentage of vulnerabilities resolved on the very first script execution attempt without retries.	Indicates the precision of the LLM's initial contextual understanding.	High: Highly accurate system prompts. Low: Weak initial context alignment.
Human Approval	PR Acceptance Rate (AR)	Ratio of human-approved and merged PRs to total submitted PRs.	Measures human trust, safety alignment, and code quality.	High: High code confidence. Low: Agent is generating risky or hallucinated fixes.
Cost Efficiency	Average Token Cost per Resolution	Mean financial cost of input/output tokens consumed per successfully resolved vulnerability.	Directly impacts operational scalability and model ROI.	Spikes indicate heavy retry loops, dense log parsing, or inefficient prompting.

12.3 Measurement Methodology & Mathematical Formulas

Metrics are mathematically calculated using the following formulations across a given evaluation window where N is the total number of unique vulnerability tickets ingested.

Resolution & Trust Formulations

- Resolution Success Rate (RSR):

$$\text{RSR} = (\text{Count of tickets marked 'Resolved' via merged PRs}) / N$$

- PR Acceptance Rate (AR):

$$\text{AR} = (\text{Approved PRs}) / (\text{Approved PRs} + \text{Rejected PRs})$$

Efficiency & Latency Formulations

- Mean Active Execution Time (MAET):

$$\text{MAET} = \text{Average of (End-to-end lifecycle time - Time spent waiting for human to merge the PR)}$$

- First-Pass Success Rate (FPSR):

$$\text{FPSR} = (\text{Count of vulnerabilities resolved on first execution without retry}) / N$$

Token and Cost Consumption

- Average Token Consumption per Vulnerability:

$$\text{Avg Tokens} = \text{Sum of (Input Tokens[i] + Output Tokens[i]) for each ticket i, divided by N}$$

13. Full Integration Configuration

13.1 System Architecture

The architecture is divided into three primary tiers: Presentation (Frontend), Orchestration (Backend/Agents), and Persistence (Database).

13.2 Presentation Tier (Frontend)

Built on Next.js, the frontend provides two primary views located in `frontend/app/`:

- Executive Dashboard (`page.tsx`): A read-only telemetry interface polling PostgreSQL via REST to display global KPIs (Average Cost per Vulnerability, Mean Time to Resolve, Success Rate, and Token Consumption).
- Remediation Command Center (`remediation/page.tsx`): The operational interface for ingesting raw scanner data (JSON/CSV) and monitoring real-time agent execution via WebSocket streams.

13.3 Orchestration Tier (Backend)

Built on FastAPI (Python 3.13+), the backend serves as the central router and agent execution environment.

- API Layer (`backend/main.py`): Exposes REST endpoints for file upload and metrics, and WebSockets for streaming agent logs.
- Agent Engine (`backend/agents/`): Utilizes LangGraph to construct cyclic, stateful graphs where specialized AI agents perform isolated tasks.

13.4 Persistence Tier (Databases)

The system employs a two-database strategy to separate volatile state from permanent analytics:

- Volatile State (SQLite): `state_db.sqlite` acts as a local queue and LangGraph checkpoint saver. It tracks active tasks (e.g., `PENDING`, `WAITING_FOR_HUMAN_APPROVAL`).
- Telemetry Warehouse (PostgreSQL): Stores finalized remediation metrics. A migration utility (`backend/utls/migrate.py`) periodically flushes resolved data from SQLite to PostgreSQL.

13.5 Multi-Agent Orchestration

The core logic of the platform relies on specialized AI agents passing shared state through a defined graph. This prevents LLM hallucination and bypasses context-window limits.

13.5.1 Agent Topology

The following agents execute in sequence, orchestrated by `main_workflow.py`:

Agent Name	Source File	Primary Responsibility
Parsing Agent	<code>parsing_agent.py</code>	Extracts raw data from uploaded Trivy/Grype/Nessus files.
Normalization Agent	<code>normalization_graph.py</code>	Standardizes disparate scanner outputs into a unified CSV format.
Asset Criticality & Lookup	<code>asset_criticality_agent.py</code> , <code>asset_lookup_agent.py</code>	Identifies the impacted system and determines business impact.
Vulnerability Intel Agent	<code>vuln_intel_agent.py</code>	Queries external APIs (NVD, EPSS) to enrich CVE data with threat context.
Prioritization Agent	<code>prioritization_graph.py</code>	Calculates a final Threat Score and sequences the execution queue.
Remediation Agent	<code>remediation_agent.py</code>	Performs Root Cause Analysis (RCA), generates a patch, and pushes a PR.

13.5.2 State Management & Human-in-the-Loop

- **Interruption:** Once the Remediation Agent successfully opens a Pull Request, it triggers an `__interrupt__` within the LangGraph state.
- **Suspension:** The task status shifts to `WAITING_FOR_HUMAN_APPROVAL` in SQLite.
- **Resumption:** A webhook listener catches GitHub merge events. The backend uses the PR ID to locate the suspended graph thread, wakes the agent, finalizes token accounting, and marks the task as `RESOLVED`.

14. Errors / Challenges Faced

- Non-CVE Vulnerability Handling – Challenges in standardized storage, retrieval, and processing of security findings lacking CVE identifiers.
- Cross-Identifier Asset Correlation – Difficulty in identifying and consolidating the same asset when different scanners provide different identifiers (e.g., IP vs. hostname).
- Sequential Remediation Queue Processing – Single-queue execution limited remediation throughput, as subsequent workflows waited for human approval of previous ones.
- Truncated CI Pipeline Error Logs When Fetching via API
- The LLM got stuck in a 3-minute tool-calling loop, spiking costs and latency, requiring intensive prompt engineering to minimize Tavily API calls.
- Integrating GitHub MCP tools with massive metadata inflated the LLM's input tokens, significantly increasing token overhead per invocation.
- GitHub webhooks fired constantly—even for the agent's own PRs and comments—making it messy to resume the workflow only when a human action is performed on PR.

15. Conclusion

Platform Transformation

Transforms a manual and non-scalable vulnerability management workflow into a low-latency, intelligence-driven, horizontally scalable zero-touch remediation pipeline where human involvement is limited to just Human-In-The-Loop validation.

Scalability

The platform is designed to process enterprise-scale workloads (~5,000+ vulnerability findings per week) through decoupled, agent-based concurrent prioritization & remediation. (the remediation-per-week count can be increased significantly with horizontal scaling of remediation agents).

Reduced Mean-Time-To-Respond (MTTR)

Automated enrichment, prioritization and orchestration reduce mean time to remediation (MTTR) to less than 2 minutes per vulnerability workflow.

Local Threat Intelligence

Cron jobs maintain near-real-time replicas of NVD, EPSS, KEV, ExploitDB and related intelligence sources, enabling enrichment batch-lookups in a few seconds from local databases.

Traceability

Jira integration provides complete asset-vulnerability lifecycle tracking, including Open, Human Review, Approved, Remediated and Closed states.

16. Future Scope

- Non-CVE Vulnerability Support – Extend the platform to support identification, storage, enrichment, and prioritization of non-CVE security findings.
- Cross-Identifier Asset Mapping – Develop mechanisms to identify and unify assets discovered through different identifiers (e.g., IP, hostname, instance ID) across multiple scanners.
- Approved Remediation Knowledge Repository – Maintain an Asset + Vulnerability → Approved Script mapping to enable fully autonomous future remediations.
- Multi-Agent Concurrent Remediation Framework – Introduce parallelized remediation agents to improve remediation throughput and reduce execution time.

17. Member Contributions

Name	Contribution
Sidharth Banthia	System Architecture & Design, Asset Criticality Engine (Business Context), Vulnerability Prioritization, Database Schema Design, Threat Intelligence Cron Jobs, Asset Enrichment Framework, Vulnerability Lab Setup, Dataset Generation, Scanner Output Deduplication
Abhimanyu Joshi	Vulnerability Prioritization, Vulnerability Enrichment Engine, Asset Criticality Research, Database Schema Design, Threat Intelligence Cron Jobs
Rahul Raj	Parser Agent, Normalization Agent, Remediation Agent, CI/CD Pipeline Development, Agent Performance Evaluation
Prince Jain	Jira Agent, Scanner Output Analysis, Prioritization Formula Research, Audit Logging & Traceability
Vaibhav Samdani	Frontend & Backend Development, Dashboard Visualization, Multi-Agent Orchestration, Normalization Agent, SQLite Design & Integration

18. References & Resources

- [Github Repository](#)
- [Workflows](#)
 1. <https://www.cisa.gov/stakeholder-specific-vulnerability-categorization-ssvc>
 2. <https://docs.rapid7.com/nexpose/working-with-risk-strategies-to-analyze-threats/>
 3. <https://www.cisa.gov/known-exploited-vulnerabilities-catalog>
 4. <https://www.first.org/epss/model>
 5. <https://nvd.nist.gov/vuln>
 6. https://gitlab.com/exploit-database/exploitdb/-/blob/main/files_exploits.csv
 7. <https://docs.github.com/en/webhooks>
 8. <https://docs.aws.amazon.com/systems-manager/latest/userguide/documents.html>
 9. <https://docs.github.com/en/actions>
 10. <https://docs.langchain.com/oss/python/langgraph/overview>
 11. <https://docs.langchain.com/>
 12. <https://developers.openai.com/api/docs>
 13. <https://ai.google.dev/gemini-api/docs>
 14. <https://cdn2.qualys.com/docs/mktg/2021/qsc/training/virtual/day-2-cybersecurity-asset-management/csam-qsc2021-slides.pdf>